HotStuff-1: Linear Consensus with One-Phase Speculation

Abstract

This paper introduces HotStuff-1, a BFT consensus protocol that improves the latency of HotStuff-2 by two network-hops while maintaining linear communication complexity against faults. Additionally, HotStuff-1 incorporates an incentive-compatible leader rotation design that motivates leaders to propose transactions promptly.

HotStuff-1 achieves a reduction of two network hops by sending clients early finality confirmations *speculatively*, after one phase of the protocol. Introducing speculation in streamlined protocols is challenging because unlike stable-leader protocols, these protocols cannot stop the consensus and recover from failures. Thus, we identify *prefix speculation dilemma* in the context of streamlined protocols; HotStuff-1 is the first streamlined protocol to resolve it.

HotStuff-1 embodies an additional mechanism, *slotting*, that thwarts delays caused by (1) rationally-incentivized leaders and (2) malicious leaders inclined to sabotage other's progress. The slotting mechanism allows leaders to dynamically drive as many decisions as possible allowed by network transmission delays before view timers expire, thus mitigating both threats.

1 Introduction

This paper introduces HotStuff-1, a BFT consensus protocol designed to reduce latency while simultaneously maintaining scalability. HotStuff-1 is primarily motivated by blockchains and online platforms that support digital asset payments and marketplaces [11, 71, 94]. These systems employ a BFT consensus protocol because it enables them to provide their clients access to a verifiable immutable ledger managed by multiple distrusting nodes, some of which may be malicious. In these systems, especially financial platforms, response latency is crucial for user engagement and satisfaction. Moreover, the demands for low response latency are posed not only by the market but also by regulation. A manuscript detailing regulatory technical requirements for Financial Market Infrastructure (FMI) states 12 key standards for operating an FMI, among which are performance requirements such as meeting peak throughput demand and timely responsiveness [1].

In this paper, we are interested in BFT consensus protocols for *partially-synchronous* setting, due to their safety against temporary network delays. Pioneering BFT consensus protocols belonging to the PBFT family [27, 44] employ a *stable-leader* design, where one replica designated as the leader initiates a two-phase consensus algorithm that determines the ledger. Unfortunately, the stable-leader design has some drawbacks. **D1:** a dedicated leader increases *censorship opportunities*, as the leader decides what transactions to propose [95]. **D2:** when the leader fails, these protocols switch to a *view-change* algorithm that incurs quadratic communication complexity to replace the leader (or change the view) and drops the system throughput to zero, as consensus on new transactions can start only after the view-change [7, 29]. **D3:** it *inhibits load and reward balancing* among the replicas [46, 89]. **D4:** a malicious leader can *keep the system throughput at the lowest level* and prevent detection as malicious (eventually replaced) by proposing transactions just before the timeout period [7, 17, 29]. Some recent protocols that follow the stable-leader design attempt to solve **D3** and **D4** by requiring all the replicas to act as the leader and/or track the leader's performance [7, 17, 29, 45, 62, 88]. However, these works require several redundant rounds of consensus that track the leader's performance (e.g. RBFT [17] and Fairledger [62]) and face collusion attacks by multiple malicious leaders (e.g. MirBFT [88] and RCC [45]).

Alternatives to the stable-leader design emerged in the blockchain world. First, Tendermint introduced a design that proactively replaces the leader at the end of each consensus decision [24]. Later, HotStuff [95] reduced view-change communication costs to linear, and additionally *streamlined* protocol phases to (at least) double throughput (solving **D1** to **D4**). Thus, streamlined linear protocols in the HotStuff family mitigate the drop in system throughput by allowing regular leader replacement at (essentially) no communication cost. However, these protocols face the following three additional challenges:

D5: Increased latency. Despite recent improvements [67], streamlined protocols incur higher latency than the stable-leader protocols that employ optimizations like *speculative-execution* [40, 42].

D6: Leader-slowness phenomenon. In blockchain systems, regular leader replacement creates an undesirable incentive structure: a leader may be inclined to delay proposing a block of transactions as close as possible to the end of its view expiration period in order to pick the transactions that offer the highest fees. Similarly, block-builders participating in a proposer-builder auction will wait as long as possible to maximize MEV (maximal extractable value) exploits [31, 74, 76]. Thus, rational leaders/builders may slow down progress and cause clients to suffer increased latency.

D7: Tail-forking attack. BeeGees [39] exposed another vulnerability of streamlined protocols, where faulty leaders prevent proposals by correct leaders from being committed unless there are consecutive correct leaders. This attack surfaces when faulty leaders are interjected between correct leaders as leaders are rotated. While they may not succeed in completely censoring transactions, faulty leaders may cause specific clients to suffer increased latency and overall, slow down progress.

Thus, we are facing a conundrum: on the one hand, stable-leader protocols yield optimal latency under no-failure cases through speculative execution and do not face **D5** to **D7**. However, they have yet to solve **D1** and **D2**, and solving **D3** and **D4** introduces new challenges. On the other hand, streamlined protocols resolve **D1** to **D4** but have yet to solve **D5** to **D7**.

HotStuff-1 resolves these seeming trade-offs by introducing a BFT consensus solution that embodies two principal contributions:

(1) A novel algorithmic core that combines regular leader rotation with linear communication, streamlining and speculative execution. HotStuff-1 acts as an optimist by speculatively executing client requests and serving the clients with the results of uncommitted transactions.
(2) An *adaptive slotting* algorithm that provides each leader with multiple slots to propose transactions. HotStuff-1 uses

slotting to maintain consistent high performance by mitigating the impacts of leader-slowness and tail-forking.

Early Finality Confirmation through Speculation. The notion of applying speculative execution to BFT protocols is not new. In his PhD thesis [26], Miguel Castro presented the idea of applying *tentative execution* to PBFT, which was later expanded/evaluated by PoE [42]. Several other flavors of speculative execution also exist, like in Zyzzyva [59] and SBFT [40]. These papers illustrate that speculative execution can help reduce the latency of BFT consensus in the good/no-failure case. Unfortunately, applying speculative execution to streamlined protocols is not a straightforward extension.

These stable-leader protocols **stop** speculative execution during the recovery/view change phases because they need to run an explicit view-change protocol (**D2**). At the end of the view-change protocol, all the replicas start the new view when they receive from the new leader a *state*. This state starts from the last agreed upon checkpoint, and for each sequence number that some replica claims to have observed since the last checkpoint, this state includes a prepare-certificate (if available) or a proposal from previous leader.[1] However, before a replica can add any of these sequence numbers/proposals to their log, the leader needs to re-run consensus on each of them.

Streamlined protocols **do not have** the option of stopping the consensus and re-running consensus on past transactions, which makes introducing speculation challenging. Thus, we identify the existence of a conundrum when applying speculation to the streamlined protocols; we term this conundrum as the *prefix speculation dilemma*. HotStuff-1 is the first streamlined protocol to employ speculative execution and resolve this conundrum by dictating when it is safe for a replica can speculatively execute a proposal.

Consequently, HotStuff-1 treats clients as first-class citizens of consensus by serving them with *early finality confirmation*. HotStuff-1 builds streamlining and speculation over HotStuff-2 [67]. Rather than forcing replicas to wait until they learn whether a transaction has committed, as in HotStuff-2, HotStuff-1 allows replicas to send commit-votes on transactions directly to clients precisely when a transaction is prepared and highly likely to commit, which also allows replicas to speculate on the execution results and send responses to clients. On collecting responses from a quorum of $\mathbf{n}-\mathbf{f}$ replicas, clients can learn two things at once: a commit decision and its execution result, which enables an early finality confirmation. Thus, HotStuff-1 helps to meet the challenges **D1** to **D5**.

Low latency through slotting. HotStuff-1 meets a subset of the challenges we listed earlier in this section, but we have yet to resolve challenges like leader slowness (**D6**) and tail-forking attacks (**D7**). Therefore, we incorporate a novel *slotting* mechanism into HotStuff-1. Slotting allows each leader to propose multiple successive blocks of transactions; each leader has access to multiple slots and can propose one block of transactions per slot. Assigning more than one slot to a leader motivates a rational leader to ensure that its blocks commit quickly, opening the opportunity to propose more new blocks. Moreover, a malicious subsequent leader can only prevent the block proposed in the last slot from committing.

[1]Alternatively, if the new leader does not have access to any prepare-certificate for a sequence number, it can leave that sequence number as empty [27].

However, having a fixed number of slots per leader/view would not eliminate the slowness attack; a fast leader will slow down its last slot. Therefore, we devise an *adaptive* slotting mechanism that allows a leader to propose as many slots as it can during the time span allotted to its view. Permitting adaptive slotting in a streamlined consensus protocol unravels a new challenge: how can the subsequent leader determine if it has received the certificates corresponding to the last slot of the preceding leader? We introduce the notion of *helper blocks* to help leaders propose their first slot without any delay.

We illustrate the practicality of our design by implementing HotStuff-1 (with and without slotting) in Apache ResilientDB (incubating) [13] and evaluating it against two baselines: HotStuff and HotStuff-2. Our results affirm that HotStuff-1 yields lower latency than the baselines; in the no-failure case, HotStuff-1 (with and without slotting) yields up to 41.5% and 24.2% lower latency. Additionally, we illustrate the resistance of HotStuff-1 (with slotting) against leader-slowness and tail-forking attacks. In summary, we make the following contributions:

(1) We introduce HotStuff-1, the first speculative, streamlined and linear BFT consensus protocol that treats clients as first-class citizens of the consensus process and serves them with early finality confirmations for their transactions.

(2) We expose a prefix speculation dilemma that exists in the context of streamlined BFT protocols that employ speculation and present a solution tailored for HotStuff-1.

(3) We introduce slotting in HotStuff-1 to mitigate leader-slowness and tail-forking attacks. Our slotting mechanism is adaptive, yet guarantees no delay for subsequent leaders.

2 Background and System Model

Modern databases require replication to guarantee availability to their clients; consensus protocols help keep these replicas consistent [61, 73]. As consensus can quickly bottleneck system performance, a large body of existing attempts to optimize these protocols [93]. A majority of existing databases [30, 52, 92] employ *crash fault-tolerant* consensus protocols to guarantee consistent replication despite crash failures [61, 73]. This crash-failure threat model is sufficient for these databases as they are managed by a single organization. In this paper, we focus on designing efficient Byzantine Fault-Tolerant (BFT) consensus protocols that can guard against arbitrary malicious failures. Such protocols are necessary for databases managed by multiple parties; commonly used in financial trading and blockchain applications [8, 13, 91].

We assume the system model adopted by existing partially synchronous BFT consensus protocols [27, 40, 42, 59, 96]. We assume a system of $\mathbf{n}$ replicas, of which at most $\mathbf{f}$ are faulty (malicious or crash-failed), and the remaining $\mathbf{n}-\mathbf{f}$ replicas are correct; $\mathbf{n} \geq 3\mathbf{f}+1$. Correct replicas follow the protocol: on the same input, produce the same output. This system receives requests from a set of clients; any number of clients can be faulty. We use R and c to denote a replica and a client, and each replica is assigned a unique identifier in the range $[1, \mathbf{n}]$ using function $\mathrm{id}(R)$.

Authenticated communication: each client/replica uses digital signatures to sign a message [55]. Additionally, replicas make use of BLS threshold signature scheme [23] to form $(\mathbf{n}, t)$ threshold

signatures. Each replica R has access to a private signature key, which it uses to create a signature share δ_R. An aggregator needs only t shares out of $\mathbf{n}$ to create the threshold signature. A receiver can use the corresponding public key to verify whether at least t replicas contributed to this signature. We use the notation $\langle m \rangle_R$ to denote a signature or a threshold signature share on message m by replica R. Correct replicas only accept *well-formed* messages that have a valid signature. Further, we assume existence of collision-resistant hash function $H(x)$, where it is impossible to find a value x', such that $H(x) = H(x')$ [55].

Adversary model: Faulty replicas can delay, drop, and duplicate any message and collude with each other. However, a faulty replica cannot forge the identity/messages of a correct replica.

Synchrony: We assume a partial synchrony model [36] where there is a known bound Δ on message transmission delays, such that after an unknown time called *GST* all transmissions arrive within Δ bound to their destinations.

System Guarantees: The goal is for replicas to form an agreement on a global ledger of transactions requested by clients, and respond to clients with the outcome of executing transactions in sequence order. There are two requirements, *safety* is required under asynchrony and *liveness* requires synchrony/*GST*:

(1) **Safety**: If two correct replicas, R and R' commit two transactions T and T' at sequence number k then $T = T'$.
(2) **Liveness**: Each correct replica will eventually commit a transaction T.

3 Speculation in Streamlined Protocols

Our primary goal is to reduce the latency for partially-synchronous streamlined consensus protocols. That is, we aim to bridge the gap between latency of streamlined protocols and optimized stable-leader consensus protocols without losing a vital tenet: linearity. An additional goal of this work is to mitigate the slowness attacks and tail-forking attacks from streamlined protocols.

To this extent, we design HotStuff-1, which uses two popular system design principles, speculation and slotting, to guarantee (1) low latency while maintaining linearity, (2) freedom from the slowness attack, and (3) in-frequent tail-forking attack.

In the rest of this section, we discuss HotStuff-1 (speculation) and defer discussion on slotting until §6. To illustrate the challenges in introducing speculation to streamlined consensus protocols, we first briefly recap the skeleton of the HotStuff-2 [67] protocol.

Recap of HotStuff-2.

HotStuff-2 optimizes HotStuff [95] by reducing commit latency by one phase (or two half-phases). Specifically, HotStuff-2 operates in a succession of *views* (Figure 1(i)). In each view, a leader proposes a transaction T and forms consecutive certificates on the initial proposal over two-and-half phases. In the first half-phase, the leader proposes transaction T. In each subsequent phase:

(1) Replicas generate threshold signature shares to ensure that at least $\mathbf{n} - \mathbf{f}$ replicas accept the leader's proposal and send it to the leader.
(2) The leader aggregates threshold shares from $\mathbf{n} - \mathbf{f}$ replicas into a threshold signature, which we refer to as a *certificate*, and broadcasts it to all the replicas.

This chain of certificates guarantees safety as follows: The first certificate (*prepare-certificate*) guarantees non-equivocation by proving that it chains to a correct previous certificate and has the support of at least $\mathbf{n} - \mathbf{f}$ replicas. The second is a *commit-certificate*, a certificate-of-certificate, guaranteeing that $\mathbf{n} - \mathbf{f}$ replicas have received the prepare-certificate, and despite any $\mathbf{f}$ failures, T will be committed. Replicas that learn the commit-certificate can mark T *committed*, execute it, and return responses to the client; T becomes committed to the immutable ledger. These responses to the clients are often referred to as *finality confirmations*, as the corresponding transactions will never get revoked.

Sending early finality confirmations.

In the good case (no-failures), a HotStuff-2 client receives finality confirmations after two and half-phases (excluding the two network hops to receive client requests and send a response to the client). With HotStuff-1, we want to cut down this delay to one and a half-phases. HotStuff-1 achieves this goal by making clients the first-class citizens of the consensus process–direct *learners* of consensus decisions. HotStuff-1 requires replicas to employ speculative execution to serve clients with early finality confirmations.

Rather than requiring replicas to wait until they learn whether a transaction *has committed*, HotStuff-1 allows replicas to speculate precisely when a transaction is prepared and highly likely to be committed by a quorum in HotStuff-2. More specifically, replicas are allowed to speculate on a proposal in the second phase of the protocol, upon voting to commit a prepare-certificate. Replicas execute a transaction T as soon as they have the prepare-certificate for T and send a response to the clients. Thus, clients directly receive commit-votes and the result of executing T, which enables an early finality confirmation. When a client receives responses from a quorum of $\mathbf{n} - \mathbf{f}$ replicas, it learns two things: a transaction has been committed, and the execution result has been prepared in advance. Safety follows from the commit-safety of HotStuff-2 because a client can determine if a commit-certificate will form.

In a non-speculative protocol, a client needs to collect only $\mathbf{f} + 1$ execution responses to determine the finality because correct replicas execute a transaction once the commit decision is reached; response from just one correct replica guarantees commitment. However, in HotStuff-1, clients need to collect $\mathbf{n} - \mathbf{f}$ responses because $\mathbf{f} + 1$ speculative responses only guarantee that one correct replica prepares the transaction. Commitment is guaranteed only when at least $\mathbf{f} + 1$ correct replicas prepare the transaction. Thus, a client learns that a transaction will finalize only upon collecting $\mathbf{n} - \mathbf{f}$ responses. Figure 1 (ii) depicts our HotStuff-1 protocol.

Although clients of HotStuff-1 need to wait for $\mathbf{f}$ additional messages (temporarily increasing memory footprint) compared to the clients of HotStuff and HotStuff-2, we argue that HotStuff-1 clients do not incur higher latency because they receive early finality confirmations. In §7, we conduct several experiments to validate this claim. Moreover, a client can delay verifying and processing these additional responses as long as needed while prioritizing other tasks. Such a delay would not impact the latency of HotStuff-1 because replicas do not wait for any input from the client.

The Prefix Speculation Dilemma.

In HotStuff-1, when clients receive a quorum of responses for a transaction (say T), they learn that T will get committed and

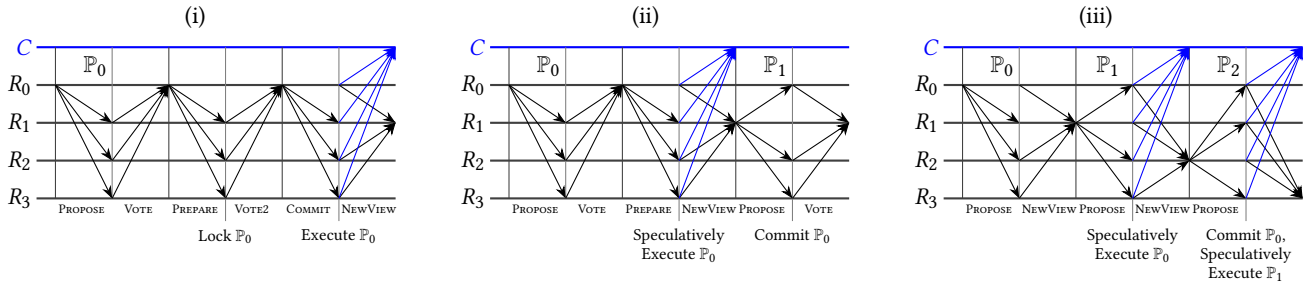

Figure 1: Workflows of (i) Basic HotStuff-2**, (ii)** Basic HotStuff-1**, and (iii)** Streamlined HotStuff-1

finality has been reached on adding T to the ledger in sequence order. The transactions preceding T in the sequence also become committed by this decision, and the result of executing T represents processing the *full prefix* of transactions up to and including T. However, the responses for T **must not** be combined with responses on transactions that precede T to form a quorum. That is, say T succeeds an earlier transaction T' in sequence order, $T' \prec T$. The commit-votes (speculative responses) of T must not be used as commit-votes of T' in forming a commit-decision on T'. See Appendix § A.1 for a detailed explanation of why this breaks *safety*.

This brings forth a challenging dilemma with respect to speculation: the responses from T represent the execution of a full prefix ending with T. When a replica R speculatively executes T, it must execute all transactions that precede it. However, if R did not commit the preceding transaction T' prior to executing T, it **must not** send responses for T' to clients because these responses represent commit-votes. Clients can mistakenly combine commit-votes from a partial quorum on T' with commit-votes from another partial quorum on T and assume that a decision has been reached on T'. On the other hand, if the replica does not send a response for T', then should T become committed, there would be a gap: the responses from T were sent, but responses from $T' \prec T$ are missing.

Note on Speculation in Stable-Leader Protocols.

As stated in the introduction, the notion of applying speculative execution to BFT protocols is not new [26, 40, 42, 59]. However, we argue that applying speculative execution to streamlined protocols is not a straightforward extension.

These stable-leader protocols stop speculative execution during the view change phases because they need to run an explicit view-change protocol. At the end of the view-change protocol, all the replicas start the new view when they receive from the new leader a *state*. This state starts from the last agreed upon checkpoint, and for each sequence number that some replica claims to have observed since the last checkpoint, this state includes a prepare-certificate (if available) or a proposal from previous leader. However, before a replica can add any of these sequence numbers/proposals to their log, the leader needs to re-run consensus on each of them.

Even though stable-leader protocols require their clients to not combine votes on a transaction across views, the ability to stop consensus, change views, and re-run consensus on past transactions ensures that there is never a situation where a replica is executing a transaction T but is yet to commit a preceding transaction T'.

Tackling Prefix Speculation Dilemma.

Streamlined protocols **do not have** the option of stopping the consensus and re-running consensus on past transactions. Thus, we state the following two rules to tackle prefix speculation dilemma in streamlined protocols:

Definition 3.1. **Prefix Speculation Rule.** A replica R can speculatively execute a transaction T if T extends a prefix which is already known to commit.

Definition 3.2. **No-Gap Rule.** A replica R receives the certificate for T that was proposed in the preceding view (i.e., consecutive).

A streamlined protocol can safely employ speculative execution if and only if it meets both of these rules simultaneously. HotStuff-1 strictly follows both of these rules.

Rollback.

Finally, we need to address the possibility that speculation does not succeed. Upon speculatively executing T, a replica R cannot commit T to the (global) ledger yet as it does not know if this transaction T will commit. Instead, each replica maintains a *local-ledger*, where it marks T prepared and executed. If in a succeeding view, R receives a prepare-certificate that does not extend T, then R must perform a *rollback* operation. This replica can observe that at least $\mathbf{n} - \mathbf{f}$ replicas decided that T cannot commit and prepared another transaction T'. Specifically, the replica should now synchronize with the other replicas to fetch the transaction T', erase T from its local-ledger, execute T', add an entry for T' to its local-ledger, and respond to the client. We discuss this in more detail in §4.2.

4 Speculative Core

We first describe the non-streamlined variant of HotStuff-1; in §5, we extend the description to the streamlined variant.

4.1 Non-Streamlined Speculation

As we treat clients as first-class citizens, we start by describing the client's behavior.

Client Request. When a client c wants the replicas to process its transaction T, it creates a Request message including T and broadcasts it to the replicas.

Client Response. When a client c receives identical Response messages from $\mathbf{n} - \mathbf{f}$ replicas for its transaction T, it records this set of responses as an early finality confirmation for T, marks T as *executed* and accepts the result of execution.

Local state (replica R) :

$\mathcal{P}(v_{lp})$, v_{lp}: stores the highest known prepare certificate and its view number
$C(v_{lc})$, v_{lc}: stores the highest known commit certificate and its view number
v: current view
$\mathcal{T}$: pending, uncommitted blocks of transactions
local-ledger, global-ledger

Figure 2: Local state on each replica of Basic HotStuff-1.

Replica pseudocode. In Figures 2 and 3, we present the pseudocode for basic HotStuff-1. Prior to describing the algorithm in detail, we lay down some useful definitions.

Definition 4.1. Prepare and Commit Certificates. A prepare-certificate $\mathcal{P}(v)$ for a proposal m aggregates $\mathbf{n}-\mathbf{f}$ threshold signature-shares for m in view v. A commit-certificate $C(v)$ for a proposal m aggregates $\mathbf{n}-\mathbf{f}$ threshold signature-shares for $\mathcal{P}(v)$ in view v.

Definition 4.2. Highest Known Certificate. A certificate $\mathcal{P}(v_{lp})$ for view v_{lp} is the highest prepare-certificate, known to replica R. For brevity, we omit from the code explicitly updating v_{lp} every time R learns a new certificate.

Definition 4.3. Extending Certificates. Given two certificates $\mathcal{P}(v)$ and $\mathcal{P}(w)$, for views v and w, at a replica R, $\mathcal{P}(v)$ extends $\mathcal{P}(w)$ if $v > w$ and $\mathcal{P}(v)$'s construction includes hash of $\mathcal{P}(w)$. Further, if a certificate $\mathcal{P}(k)$ extends $\mathcal{P}(v)$ and $\mathcal{P}(v)$ extends $\mathcal{P}(w)$, then transitively $\mathcal{P}(k)$ extends $\mathcal{P}(w)$.

Definition 4.4. Conflicting Certificates. Given two certificates, $\mathcal{P}(v)$ and $\mathcal{P}(w)$, for views v and w, at a replica R, $\mathcal{P}(v)$ conflicts with $\mathcal{P}(w)$ if neither $\mathcal{P}(v)$ extends $\mathcal{P}(w)$, nor $\mathcal{P}(w)$ extends $\mathcal{P}(v)$.

Local state at a replica includes: (1) highest prepare-certificate, $\mathcal{P}(v_{lp})$, formed in view v_{lp}, (2) highest commit-certificate, $C(v_{lc})$, formed in view v_{lc}, (3) current view v, (4) set of pending, uncommitted blocks of transactions $\mathcal{T}$, and (5) the local-ledger and the global-ledger.

Propose. When the leader $\mathcal{L}_v$ for view v–a replica R with $v = \mathrm{id}(R) \bmod \mathbf{n}$–enters view v, it waits to receive NewView messages from at least $\mathbf{n}-\mathbf{f}$ replicas. Each message carries the highest certificate known to its sender, which helps the leader learn the highest known certificate among them and update its v_{lp}. Additionally, if these $\mathbf{n}-\mathbf{f}$ NewView messages contain threshold signature-shares for the highest $\mathcal{P}(w)$, the leader forms a commit-certificate $C(w)$ (Figure 3, Line 7) and updates $C(v_{lc})$. Next, the leader aggregates client transactions (yet to be proposed) into a block B_v and creates a Propose message that includes the view number v, B_v, $\mathcal{P}(v_{lp})$, and $C(v_{lc})$ (if formed). Then, $\mathcal{L}_v$ broadcasts this Propose message (lets call it m) to all the replicas (Lines 4-5). *Note.* The Propose message for view 0, the genesis view, extends a hard-coded certificate that all replicas assume to be valid.

ProposeVote. On receiving a Propose message m from the leader (Line 11), a replica R checks if one of the following holds:

(1) The prepare certificate $\mathcal{P}(w)$ in m matches the highest prepare-certificate $\mathcal{P}(v_{lp})$ of view v_{lp} at R, or

(2) $w > v_{lp}$, in which case R updates its v_{lp} to w, sets $\mathcal{P}(w)$ as highest known prepare certificate and runs the *recovery protocol* to fetch the block corresponding to $\mathcal{P}(w)$ (§4.2).

Leader role (running at leader $\mathcal{L}_v$) :

1: **event** Upon pacemaker.EnterView(v) **do**
2: Wait until received $\mathbf{n}-\mathbf{f}$ NewView messages for view v
3: Wait until $v_{lp} == v-1$ or pacemaker.ShareTimer(v)
4: Let B_v be a block of client transaction yet to be proposed
5: Broadcast $m = \langle \text{Propose}, B_v, v, \mathcal{P}(v_{lp}), C(v_{lc})\rangle_{\mathcal{L}_v}$

6: **event** Received $\mathbf{n}-\mathbf{f}$ NewView messages with shares of $C(w)$ **do**
7: $C(w) \leftarrow$ CreateThresholdSign($\mathbf{n}-\mathbf{f}$ distinct δ_R^C shares)

8: **event** Received $\mathbf{n}-\mathbf{f}$ ProposeVote messages **do**
9: $\mathcal{P}(v) \leftarrow$ CreateThresholdSign($\mathbf{n}-\mathbf{f}$ distinct $\delta_R^{\mathcal{P}}$ shares)
10: Broadcast $\langle \text{Prepare}, v, \mathcal{P}(v)\rangle_{\mathcal{L}_v}$

Backup role (running at each replica R (including leader)) :

11: **event** Received $\langle \text{Propose}, b_v, \mathcal{P}(w), C(x)\rangle_{\mathcal{L}_v}$ **do**
12: Execute all transactions up to (incl.) B_x, add result to global-ledger and respond to clients ▹ *traditional-commit rule*

13: **if** $w \geq v_{lp}$ ▹ *vote to prepare B_v* **then**
14: $\delta_R^{\mathcal{P}} \leftarrow$ CreateThresholdShare($\mathcal{P}(w), Hash(B_v)$)
15: Send $\langle \text{ProposeVote}, v, \delta_R^{\mathcal{P}}\rangle_R$ to $\mathcal{L}_v$

16: **event** Received $\langle \text{Prepare}, v, \mathcal{P}(v)\rangle_{\mathcal{L}_v}$ **do**
17: **if** $\mathcal{P}(v)$ extends $\mathcal{P}(v-1)$ ▹ *prefix-commit rule* **then**
18: Execute all transactions up to (incl.) B_{v-1}, add result to global-ledger and respond to clients

19: **if** predecessor of B_v is in global-ledger ▹ *Prefix Speculation rule* **then**
20: **if** local-ledger state conflicts with B_v **then**
21: Roll local-ledger back to the common ancestor
22: Execute all transactions in B_v speculatively, add result to local-ledger and send client a response ▹ *speculatively execute B_v*

23: $\delta_R^C \leftarrow$ CreateThresholdShare($\mathcal{P}(v_{lp})$) ▹ *vote to commit B_v*
24: Send $\langle \text{NewView}, v+1, \mathcal{P}(v_{lp}), \delta_R^C\rangle_R$ to $\mathcal{L}_{v+1}$
25: Call exitView()

26: **event** Upon timeout **do**
27: Send $\langle \text{NewView}, v+1, \mathcal{P}(v_{lp}), \perp\rangle_R$ to $\mathcal{L}_{v+1}$.
28: Call exitView()

29: **function** exitView() **do**
30: $v \leftarrow v+1$. ▹ *disable voting and speculative execution for view v*
31: Call pacemaker.completedView()

Figure 3: Basic HotStuff-1.

If neither condition is satisfied, R ignores the message. Otherwise, R creates a ProposeVote message, which includes a threshold signature-share $\delta_R^{\mathcal{P}}$ for m (includes hash of B_v), and sends this message to the leader $\mathcal{L}_v$ (Lines 11-15).

Prepare. When $\mathcal{L}_v$ receives $\mathbf{n}-\mathbf{f}$ well-formed ProposeVote messages for its proposal m, it tries to combine their signature shares into a threshold signature to create a prepare-certificate $\mathcal{P}(v)$ for view v (Lines 8-9). If successful, the leader creates a Prepare message including the certificate $\mathcal{P}(v)$ and broadcasts it (Line 10).

Vote and Speculate on Prepare. On receiving a Prepare message from the leader, a replica R checks if the certificate $\mathcal{P}(v)$ is valid, i.e., the threshold signature combines signature-shares from $\mathbf{n}-\mathbf{f}$ replicas for the leader's proposal m. If it is valid, R updates its highest known prepare-certificate to $\mathcal{P}(v)$; sets $v_{lp} = v$.

If B_v's predecessor is already in the global-ledger (i.e., meets the Prefix Speculation rule) and B_v was prepared in view v (i.e., meets the No Gap rule[2]), R does the following (Lines 16-25):

(1) *Speculatively executes* the transactions in block B_v of m.
(2) Replies to the respective clients.
(3) Adds result of executing B_v to its local-ledger.

Note on execution model. Once the transactions are ordered, they are executed sequentially. As in prior works [4, 67], this paper focuses on reducing client latency caused by consensus. Thus, we assume the simplest execution model: sequential execution of the ordered transactions. Alternatively, other execution designs, such as parallel transaction execution, can be employed, but these require detecting and resolving conflicts among transactions.

ExitView and NewView. A replica R is ready to exit view v in two cases: upon receiving a prepare message from the leader and upon a timer expiration. Prior to calling the ExitView() function, R constructs a NewView message, which includes $\mathcal{P}(v_{lp})$, and forwards it to the leader $\mathcal{L}_{v+1}$ for view $v+1$ (Line 30). It then invokes the pacemaker to orchestrate view-synchronization as needed.

Commit. There are two commit rules in basic HotStuff-1 (*traditional commit* and *prefix commit*), which dictate when a replica can write a block of transactions to the global-ledger.

Definition 4.5. Traditional Commit Rule. A replica marks a block B_{v-1} as committed, when it receives a commit-certificate $C(v-1)$ for B_{v-1} with a proposal for higher view $\geq v$.

Definition 4.6. Prefix Commit Rule. A replica marks a block B_{v-1} as committed when in view v, it receives a prepare-certificate $\mathcal{P}(v)$, for proposal m by $\mathcal{L}_v$, that extends $\mathcal{P}(v-1)$.

As the name suggests, the traditional commit rule is common to any consensus protocol and has been used by all the protocols of the HotStuff family. Post speculatively executing the transaction, each replica creates a threshold share (δ_R^C) on the prepare-certificate and forwards this threshold share with the NewView message to the leader of the next view (Lines 23-30). Next, each replica calls the ExitView procedure. On receiving $\mathbf{n}-\mathbf{f}$ threshold shares for the same prepare-certificate, the leader of the next view combines them into a commit-certificate and forwards it to all the replicas (Line 7). Upon receiving the commit-certificate $C(v)$, a replica R adds the block B_v to the global-ledger and marks it committed (Line 12). *Note:* on receiving the commit-certificate, R replies to a client if R had not sent a speculative response for this transaction.

The prefix commit rule is an important optimization that allows correct replicas to commit blocks when HotStuff-1 is experiencing replica failures. We will expand on this in the next section.

Example. Assume that the pacemaker (§4.2.1) informs the replicas that they are in view v. Let R_0 (Figure 1(ii)) be the leader for view v. R_0 aggregates some client requests to form a block B_v, creates a Propose message that includes the highest certificate $\mathcal{P}(v-1)$ and the commit-certificate $C(v-2)$ (composed of the threshold signatures for view $v-2$), and broadcasts this message to all replicas. Upon receiving the proposal (say m) from the leader of view v, R_0, all replicas verify that $\mathcal{P}(v-1)$ is the highest certificate. They then send a ProposeVote message to R_0 that includes a threshold signature share for m. R_0 combines the $\mathbf{n}-\mathbf{f}$ threshold signature shares from the ProposeVote messages to create a prepare-certificate $\mathcal{P}(v)$ and forwards this to all replicas as a Prepare message. Each replica, upon receiving a Prepare message from R_0, speculatively executes B_v, replies to the clients, and sends a NewView message to R_1 (the leader for the next view), which includes a threshold signature share for $\mathcal{P}(v-1)$ and the highest certificate $\mathcal{P}(v)$.

[2]We define No Gap rule for streamlined protocols. However, it can be expanded to the non-streamlined versions too: it refers to the preceding phase in the same view.

4.2 Failures and Recovery Design

A malicious replica can impact the consensus in various ways if it is the leader of an ongoing view: (1) drop, delay, or prevent sending messages and/or certificates to prevent replicas from making progress, and (2) create two proposals that extend the same certificate to prevent replicas from having the same state. HotStuff-1 should quickly detect these failures and resolve them to prevent performance degradation.

Detecting lack of progress: Timeouts

Like other protocols in the partial synchrony setting, HotStuff-1 requires replicas to set timers. A replica R starts a timer following the rules defined by the *pacemaker* protocol (§4.2.1). Upon timeout, a replica R assumes that the leader of the current view (say v) has failed and thus sends a NewView message to the leader of view $v+1$. Post this, R calls the ExitView procedure to move to the next view (Lines 26-28).

Lack of certificates of the last view

Leader $\mathcal{L}_v$ of view v may fail to receive the prepare-certificate $\mathcal{P}(v-1)$ due to an unreliable network or faulty behaviors of the the preceding leader. If it extends some other lower certificate (at a view $< v-1$), its new proposal will get ignored by correct replicas that received and set $\mathcal{P}(v-1)$ as the highest known prepare-certificate, and thus cannot form the prepare-certificate $\mathcal{P}(v)$. To ensure that the new proposal is accepted by all correct replicas, $\mathcal{L}_v$ should wait for sufficiently long to receive the highest certificates known to all the correct replicas. Following the rules defined by the *pacemaker* protocol (§4.2.1), it is guaranteed that $\mathcal{L}_v$ will receive the highest certificates after pacemaker.ShareTimer(v) (Line 3).

Conflict Resolution: Rollback

When a replica R receives a prepare-certificate $\mathcal{P}(v)$ for a message m, HotStuff-1 allows R to set $\mathcal{P}(v)$ as the highest known certificate and speculatively execute transactions of block B_v in m. A faulty leader may not send $\mathcal{P}(v)$ to other replicas, in which case B_v may not get committed. To ensure replicas have a common state (global-ledger), HotStuff-1 supports state rollback (or *erasing local-ledger*). See Appendix § A.2 for a scenario illustrating this.

Definition 4.7. Rollback Condition. Given two blocks B_w and B_v, if a replica R speculatively executes transactions in B_w with prepare-certificate $\mathcal{P}(w)$ in view w, R rolls back B_w if R receives a conflicting prepare-certificate $\mathcal{P}(v)$ in view v, such that $w < v$.

Definition 4.7 tells a replica when it should rollback (or erase) its local-ledger. When a replica R receives a prepare-certificate $\mathcal{P}(w)$ in view w for a proposal m, it speculatively executes m's transactions and only updates its local-ledger; R does not add m to the global-ledger as it has only received $\mathcal{P}(w)$ for m and has no guarantee that m will commit in the future. Thus, R can erase its local-ledger

and rollback the effects of m's transactions if it receives a certificate $\mathcal{P}(v)$ for a conflicting proposal m' in view $v, v > w$ (Lines 20-21).

Prefix Commit: Processing Delayed Certificates

Due to failures, replicas may vote on a proposal in a view but not receive a prepare-certificate for that proposal in the same view. For example, the leader of view v fails before broadcasting the prepare-certificate $\mathcal{P}(v)$ for its proposal m to at least $\mathbf{n}-\mathbf{f}$ replicas. If such is the case, neither the client will receive an early finality confirmation for m, nor the replicas will receive a commit-certificate for m in view $v+1$. So, how can we decide the fate of m.

If m conflicts with another proposal m' proposed in a view $w, w > v$, then it will be rolled back as described earlier. However, if there are no conflicts, that is, the leader of some view $x, x > v$ observes $\mathcal{P}(v)$ and sets $\mathcal{P}(v)$ as the highest known certificate for its proposal m', a replica R will execute transactions in B_v and reply to the client once R receives a commit-certificate $C(x)$ for m' (Line 12).

Fortunately, we have an *optimization* that allows replicas to execute and commit B_v at least one phase earlier; if $x = v+1$, then a replica R can execute transactions in B_v, add them to the global-ledger, and reply to their clients (Line 17). We refer to this optimization as the prefix-commit rule.

Recovery Mechanism

A faulty leaders can skip broadcasting a certificate to all the replicas. If any future leader has access to this valid certificate, it can extend its new proposal from this certificate. Such scenarios can occur in any protocol of the HotStuff family and are not limited to just malicious attacks; for example, in HotStuff-2, a leader can crash fail before broadcasting the certificate to all the replicas.

If the leader $\mathcal{L}_v$ for v extends its proposal m with the certificate $\mathcal{P}(w)$ for view $w, w < v$ and forwards this as a Propose message to all replicas, then any replica R that receives this Propose message needs to validate $\mathcal{P}(w)$ and requires access to the corresponding proposal (say m') proposed in view w. If R does not have access to m', then it can fetch it from $\mathcal{L}_v$, which $\mathcal{L}_v$ should have because its proposal extends $\mathcal{P}(w)$.

4.2.1 **Pacemaker.** For a system to make *progress*, at least $\mathbf{n}-\mathbf{f}$ correct replicas should be in the same view. Otherwise, a leader cannot collect enough votes to make progress and to generate a prepare-certificate (§5). Specifically, under an unreliable network or when the leader is faulty, correct replicas can diverge: some replicas may have progressed to higher views, while others are stuck on an old view. To prevent this divergence among correct replicas, we adopt the *pacemaker* designs of prior works [28, 63]; group views into *epochs*, each of which contains $\mathbf{f}+1$ consecutive views.

In Figure 4, we illustrate the pseudocode for pacemaker. Every time a replica R reaches at the end of a view, it calls the function CompletedView (Lines 3-7) to check if the next view (say v) is part of the current epoch. If this is the case, R enters view v. Otherwise, v is the first view of the next epoch ($v \mod (\mathbf{f}+1) = 0$) and R must synchronize its view with the other replicas. R calls the function SynchronizeView(v) (Lines 8-10) and delays entering the view v until the view synchronization is complete.

The function SynchronizeView(v) requires R to send a Wish message to the $\mathbf{f}+1$ leaders of the next epoch; $\mathcal{L}_{v+k}$, where $k = 0, 1, 2, \ldots, \mathbf{f}$. When a leader of the next epoch receives $\mathbf{n}-\mathbf{f}$ Wish messages for view v, it creates a *Timeout Certificate* TC_v and broadcasts it to all the replicas (Lines 14-15). Any non-leader replica R that receives TC_v forwards this certificate to all the $\mathbf{f}+1$ leaders for the next epoch. Next, R sets the *starting time* for each of the next $\mathbf{f}+1$ views $v+k, k = 0, 1, 2, \ldots, \mathbf{f}$. Say, R received TC_v at time t, then view $v+k$ starts at time $t+k\tau$, where τ is a predetermined timer length that is sufficiently long for a non-faulty leader to reach a consensus on the proposal of its view. *Note:* the starting time for view $v+k$ is also the timeout for view $v+k-1$. Post this, R enters the next view v (Lines 16-18).

```
1:  function ShareTimer(v) do
2:      return StartTime[v] + 3Δ

3:  function CompletedView() do
4:      if v mod (f + 1) ≠ 0 then
5:          Call EnterView(v)
6:      else
7:          Call SynchronizeEpoch(v)

8:  function SynchronizeEpoch(v) do
9:      δ_R ← CreateThresholdShare(v)
10:     Send ⟨Wish(v, δ_R)⟩_R to leaders L_{v+k}, k = 0, 1, 2, ..., f.

    Epoch Leader role (running at leader L_{v+k}, k = 0, 1, 2, ..., f.) :
11: event Upon receiving n − f Wish messages of view v do
12:     TC_v ← CreateThresholdSignature(n − f distinct δ_r shares)
13:     Broadcast TC_v.

    Epoch Backup role (running at each replica R) :
14: event Upon receiving TC_v at time t do
15:     Relay TC_v to the leaders L_{v+k}, k = 0, 1, 2, ..., f
16:     for k ← 0, 1, 2, ..., f do
17:         StartTime[v + k] ← t + kτ
18:     Call EnterView(v)
```

Figure 4: Pseudocode of Pacemaker Protocol

Such a *pacemaker* guarantees that, after *GST*, once the first synchronization is done at view v_s, if a correct replica sets it timer for view $v, v \geq v_s$ to expire at time $t+\tau$, then all correct replicas will enter view v before $t+2\Delta$ and no correct replica will time out and enter view $v+1$ before $t+\tau-2\Delta$, where Δ is the transmission delay bound [28, 63]. If the leader $L_{v'}$ for view v' waits for an additional message delay, Δ, after $StartTime[v'] + 2\Delta$, then it is guaranteed to receive NewView messages from all the correct replicas and learn the highest known certificate. Thus, the function ShareTimer(v) returns after $StartTime[v'] + 3\Delta$.

5 Streamlined Speculation

Basic HotStuff-1 (§4.1) processes only one proposed batch of transactions every two phases. Like HotStuff, we can *streamline the phases* of HotStuff-1 to ensure that we rotate leaders and inject a new batch of transactions in each phase. This has the potential to increase throughput by 2×.

Borrowing from the streamlined variant of HotStuff, streamlined HotStuff-1 works as follows: it overlaps the second phase of view v, consisting of *Prepare* and *NewView* steps, with the first phase of view $v+1$, namely, *Propose* and *ProposeVote* steps. Each view (or leader) lasts for only one phase. The leader of each view waits for $\mathbf{n}-\mathbf{f}$ NewView messages from the preceding view. The leader first

Leader role (running at leader $\mathcal{L}_v$) :

1: **event** Upon PACEMAKER.ENTERVIEW() **do**
2: Wait until received $\mathbf{n} - \mathbf{f}$ NEWVIEW messages for view v
3: Wait until $v_{lp} == v - 1$ or SHARETIMER(v)
4: Let B_v be a block of client transaction yet to be proposed
5: Broadcast $m = \langle \text{PROPOSE}, B_v, v, \mathcal{P}(v_{lp}) \rangle_{\mathcal{L}_v}$

6: **event** Received $\mathbf{n} - \mathbf{f}$ NEWVIEW messages with shares of $\mathcal{P}(w)$ **do**
7: $\mathcal{P}(w) \leftarrow$ CREATETHRESHOLDSIGN($\mathbf{n} - \mathbf{f}$ distinct δ_R shares)

Backup role (running at each replica R (including leader)) :

8: **event** Received $\langle \text{PROPOSE}, B_v, \mathcal{P}(w) \rangle_{\mathcal{L}_v}$ **do**
9: **if** $\mathcal{P}(w)$ extends $\mathcal{P}(w-1)$ ▹*commit-rule* **then**
10: Execute all transactions up to (incl.) B_{w-1}, add result to global-ledger and respond to clients

11: **if** $w == v - 1$ ▹*No-Gap rule* **then**
12: **if** predecessor of $\mathcal{P}(v-1)$ is in global-ledger ▹*Prefix Speculation rule* **then**
13: **if** local-ledger state conflicts with B_{v-1} **then**
14: Rollback local-ledger to the common ancestor
15: Execute all transactions in B_{v-1} speculatively, add result to local-ledger and send client a response ▹*speculatively execute B_{v-1}*

16: **if** $w \geq v_{lp}$ **then**
17: $\delta_R \leftarrow$ CREATETHRESHOLDSHARE($\mathcal{P}(v_{lp}), Hash(B_v)$)
18: Send $\langle \text{NEWVIEW}, v+1, \mathcal{P}(v_{lp}), \delta_R \rangle_R$ to $\mathcal{L}_{v+1}$
19: Call EXITVIEW()

20: **event** Upon timeout **do**
21: Send $\langle \text{NEWVIEW}, v+1, \mathcal{P}(v_{lp}), \perp \rangle_R$ to $\mathcal{L}_{v+1}$.
22: Call EXITVIEW()

23: **function** EXITVIEW() **do**
24: $v \leftarrow v + 1$. ▹*disable voting for view v*
25: Call PACEMAKER.COMPLETEDVIEW()

Figure 5: Streamlined HotStuff-1.

attempts to create a prepare-certificate from the threshold shares it received from the replicas. It then selects the highest prepare-certificate it knows and references it in a new proposal carrying a new batch of client transactions.

Commit Rule. Unlike the basic HotStuff-1, the streamlined design has only one commit rule: replicas follow the prefix commit rule (Definition 4.6) to add a transaction to the global-ledger. As each view consists of one phase, there is no explicit opportunity to create a commit-certificate. In view v, a replica R commits a block B_{w-1}, proposed in view $w-1$, if the proposal for view v includes the certificate $\mathcal{P}(w)$ that extends the certificate $\mathcal{P}(w-1)$. *Note:* We no longer distinguish between prepare and commit certificates as in basic HotStuff-1.

Prefix Speculation Rule and No-Gap Rule.

As in the basic variant, rules guaranteeing safe speculation are needed in streamlined HotStuff-1 to tackle the Prefix Speculation dilemma described in §3. The enforcement of the Prefix Speculation rule is similar to the basic regime: *a replica R can speculate on a block B_v provided that $\mathcal{P}(B_v)$ extends a certificate $\mathcal{P}(B_w)$, where B_w is committed.* See Appendix § A.3 for an example of not following Prefix Speculation rule in streamlined HotStuff-1. Similarly, enforcement of No Gap rule (Definition 3.2) is necessary, that is, $w = v - 1$.

5.1 Streamlined HotStuff-1 Protocol

The streamlined protocol is reduced into a single phase of (1) *propose* and (2) *vote* that includes the speculative execution as demonstrated in Figure 1(iii).

Propose. When the leader $\mathcal{L}_v$ for view v–the replica R with $v = \text{id}(R) \bmod \mathbf{n}$–receives well-formed NEWVIEW messages from at least $\mathbf{n} - \mathbf{f}$ replicas, it tries to combine their threshold signature-shares into a threshold signature to create a certificate $\mathcal{P}(w)$ for view w, where $w < v$. If $\mathcal{L}_v$ is successful, it updates its highest known certificate $\mathcal{P}(v_{lp})$. Otherwise, $\mathcal{P}(v_{lp})$ is updated with the highest certificate carried by the $\mathbf{n} - \mathbf{f}$ NEWVIEW messages. In either case, the leader extends its highest certificate to form its new proposal as a PROPOSE message and broadcasts it to all the replicas. This proposal includes the view number v, a block B_v of client transactions yet to be proposed, and $\mathcal{P}(v_{lp})$.

Execute and Ledger Update. On receiving a PROPOSE message (lets call it m) from the leader (Line 8), a replica R checks if the view w, for the certificate $\mathcal{P}(w)$ in m, is less than v_{lp}, in which case R ignores the message. Otherwise, R sets $\mathcal{P}(w)$ as the highest known certificate $\mathcal{P}(v_{lp})$, and if R had speculatively executed transaction at v_{lp}, then it rollbacks its state.

Next, R does the following (Lines 9-19):

(1) Following the commit-rule: if $\mathcal{P}(w)$ extends $\mathcal{P}(w-1)$, then R executes transactions for all blocks up to B_{w-1} (blocks that B_{w-1} extends) if yet to be executed, adds them to the global-ledger and sends a reply to respective clients.

(2) If $w = v - 1$ (i.e., meets the No-Gap rule), then following the Prefix Speculation Rule: if the block extended by B_w has been committed, then R *speculatively executes* the transactions in blocks B_w, adds them to the local-ledger and sends a reply to respective clients.

(3) Finally, R creates a NEWVIEW message, which includes a threshold signature-share δ_R for m (includes hash of $\mathcal{P}(v_{lp})$), and sends this message to the leader of the next view, $\mathcal{L}_{v+1}$.

Timer expiration. In case of timer expiration, replica R constructs a NEWVIEW message, which includes an empty threshold signature-share and the highest known certificate $\mathcal{P}(v_{lp})$, and forwards it to the leader $\mathcal{L}$ for view $v+1$.

ExitView and NewView. Like earlier, a replica R is ready to exit view v in two cases: upon receiving a PROPOSE message from the leader and upon a timer expiration. EXITVIEW() invokes the pacemaker to orchestrate view-synchronization as needed.

Correctness Proof. See Appendix § B for correctness proof.

6 Slotting

Rotating leaders in BFT protocols leads to the following challenges:

(1) **Leader-slowness phenomenon.** Rational leaders, who are not malicious but aim to maximize their gains, may delay proposing a block of transactions until as late as possible in their rotation, as they are incentivized to include transactions that yield higher fees. Similarly, block builders participating in a proposer-builder auction may also delay to maximize MEV (maximal extractable value) exploits [31, 74, 76].

If a leader/builder proposes its block too early, it risks filling the block with transactions that offer lower fees than those that may

come in the future. Thus, rational leaders and builders may slow down progress, causing clients to experience increased latency.

Example 6.1. Assuming that each block can include at most 100 transactions and the maximum allowed time for a view to complete is 4s, while it takes a leader approximately 1s to create a block and ensure that its proposal completes all phases of HotStuff-1. In an ideal case, the latency for each transaction would be $\approx 1s$. A rational leader will wait for four seconds to create the block in the hope of selecting the top 100 highest fees paying transactions, which ensures the average latency to be $\approx 4s$.

(2) **Tail-forking attack.** In streamlined protocols, the two protocol phases necessary to commit a transaction are spread across the reign of two leaders. The second leader, if malicious, may skip the proposal by the previous leader by pretending it did not receive enough votes for it instead of helping drive it to a commit decision.

Example 6.2. Assuming that R_0 and R_1 are the leaders for views v and $v+1$, respectively, and R_1 is malicious. In view v, R_0 broadcasts a Propose message for B_v containing $\mathcal{P}(v-1)$, and all replicas send a ProposeVote message for B_v to R_1. As R_1 is malicious, in view $v+1$, assume that R_1 initiates the tail-forking attack by ignoring the NewView messages for B_v and broadcasts a Propose message for B_{v+1} that includes the certificate $\mathcal{P}(v-1)$. Since no replica has access to a higher known certificate than $\mathcal{P}(v-1)$, all replicas accept B_{v+1}, create a threshold signature-share for B_{v+1}, and send it with a NewView message to R_2. Consequently, all the work done during view v is a waste.

We resolve these two challenges by adding *slotting* to the core of streamlined consensus protocols. Slotting provides each leader with opportunities to propose multiple blocks, one per slot, until their rotation time. Each leader incorporates an adaptive slotting mechanism to propose as many slots as possible within the allotted view timer. Assigning multiple slots to each leader/view: (1) motivates the leader to propose available transactions promptly rather than wait for transactions to arrive in the future because the greater the number of blocks a leader proposes, the higher reward it earns; and (2) eliminates opportunities for tail-forking attacks for all but the last slot in each view because each leader can prevent its slot to be forked as it also proposes the extending slot. With slotting, assuming Example 6.1, we expect each leader to propose at least four blocks (one per slot) per view with latency $\approx 1s$; assuming Example 6.2, R_1 can only tail-fork the last slot of view v, but three out of four blocks will reach consensus. *Note.* The benefits of slotting are only visible under heavy load—when there is an ample number of client requests to propose.

6.1 Slotting Design

We proceed to describe how to incorporate a slotting design into streamlined HotStuff-1. *Note:* Our design of slotting is applicable to any protocol of the HotStuff family.

We introduce two additional notations:

First, we enumerate leader proposals with a pair of numbers: a leader/view number and a slot number within the view. Blocks are ordered lexicographically: if $v < v'$, then block $B_{i,v}$ is ordered before $B_{i',v'}$. If $v = v'$ and $i < i'$, then block $B_{i,v}$ is ordered before $B_{i',v'}$. For instance, in Figure 6, we illustrate a chain of blocks generated

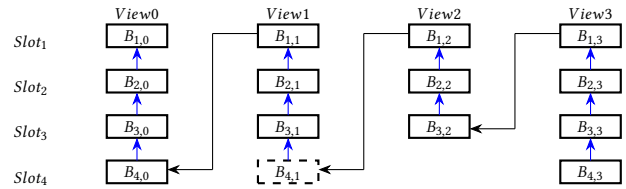

Figure 6: Chain in HotStuff-1 with Slotting. The dashed block is an uncertified helper block.

under the slotting design. Each block extends a certificate of the preceding one, resulting in a *snake-like* chain that threads blocks within each view and, at the end of each view, threaded to the next view. In the Figure, block $B_{2,1}$ includes a certificate for $B_{1,1}$, $B_{1,2}$ of $B_{4,1}$, and so on. *Second*, we introduce a new message type, NewSlot, to distinguish between when a new leader starts a new view, and when the current leader starts a new slot.

Next, we describe the protocol modifications needed to support slotting.

The local state at a replica includes slot state (refer to Figure 7), in addition to view state: (1) highest known certificate, $\mathcal{P}(s_{lp}, v_{lp})$, formed in view v_{lp}, slot s_{lp} (2) current view v and current slot s (3) the highest signature share δ_h it sent. (4) the hash the highest block it voted. As before, a replica maintains pending, uncommitted blocks of transactions, a local-ledger and the global-ledger.

Figure 8 and 9 illustrates the pseudocode of streamlined HotStuff-1 with slotting.

Propose. At each slot s, the leader $\mathcal{L}_v$ for view v awaits messages from at least $\mathbf{n}-\mathbf{f}$ replicas of either of the following types:

(1) well-formed NewView messages for view $v-1$, if $s=1$, or
(2) well-formed NewSlot messages for slot $(s-1, v)$ if $s>1$.

Thus, $\mathcal{L}_v$ administers two types of transitions.

NewView: The first is entering a new view. The leader awaits well-formed NewView messages for view $v-1$ from at least $\mathbf{n}-\mathbf{f}$ replicas. If all messages are non-empty and contain threshold signature-shares of the same slot $s^*, s^* \geq 1$, of some previous view v^*, the leader can combine these shares into a threshold signature to create a certificate $\mathcal{P}(s^*, v^*)$. Then, if the highest certificate known to $\mathcal{L}_v$ is of view $v-1$, $\mathcal{L}_v$ checks if there is any valid "helper" block.

Definition 6.3. **Helper Block B_{helper} of view v.** An uncertified block $B_{s_{lp}+1,v-1}$ that was received by $\mathcal{L}_v$ whose highest certificate $\mathcal{P}(s_{lp}, v_{lp})$ is of view $v-1$, namely, $v_{lp} = v-1$.

For example, in Figure 6, $B_{4,1}$ is an uncertified helper block contained in $B_{1,2}$, where the $\mathcal{L}_2$'s highest certificate is $\mathcal{P}(3,1)$ when $\mathcal{L}_2$ is constructing $B_{1,2}$.

If $v_{lp} = v-1$ and there is a unique helper block $B_{s_{lp}+1,v-1}$ known to $\mathcal{L}_v$ (Line 3), $\mathcal{L}_v$ proposes $B_{1,v}$ including not only a transaction T and the highest certificate $\mathcal{P}(s_{lp}, v-1)$ but also B_{helper} being $B_{s_{lp}+1,v-1}$. Otherwise, when other conditions are met (which will be explained in detail in § 6.2), $\mathcal{L}_v$ proposes $B_{1,v}$ including transaction T, the highest certificate $\mathcal{P}(s_{lp}, v_{lp})$ and an empty B_{helper} being $\perp$.

NewSlot: For slot (s, v), where $s > 1$, the leader $\mathcal{L}_v$ awaits well-formed NewSlot messages from at least $\mathbf{n}-\mathbf{f}$ replicas for $B_{s-1,v}$. Once it collects enough votes from the previous slot, it combines

Local state (replica R) :

$\mathcal{P}(s_{lp}, v_{lp})$, s_{lp}, v_{lp}: always keeps the highest known prepare certificate and its slot number and view number

s: the slot in the current view

δ_h: always keeps the highest signature-share sent

H_h: always keeps the hash of the highest voted block

Figure 7: Additional Local State in Streamlined-HotStuff-1 with Slotting.

Leader role (running at leader $\mathcal{L}_v$) :

1: **event** Upon PACEMAKER.ENTERVIEW() **do**
2: Keep updating (s_{lp}, v_{lp}) until received $\mathbf{n}-\mathbf{f}$ NEWVIEW messages
3: **if** $v_{lp} = v - 1$ **and** R knows a unique $B_{s_{lp}+1,v-1}$ **then**
4: $B_{helper} \leftarrow B_{s_{lp}+1,v-1}$
5: **else if** $v_{lp} < v - 1$ **or** R knows no $B_{s_{lp}+1,v-1}$ **then**
6: $B_{helper} \leftarrow \perp$
7: **else if** R knows multiple conflicting $B_{s_{lp}+1,v-1}$ **then**
8: $B_{helper} \leftarrow \perp, s_{next} \leftarrow s_{lp} + 1$
9: Wait until (1) updated $\mathcal{P}(s_{lp}, v_{lp})$ to $\mathcal{P}(s_{next}, v-1)$ **or** (2) $\mathbf{n} = 3\mathbf{f} + 1$ NEWVIEW messages for view v are received **or** (3) SHARETIMER(v)

10: Let $B_{1,v}$ be a block of client transactions yet to be proposed
11: Broadcast $m = \langle \text{PROPOSE}, B_{1,v}, 1, v, \mathcal{P}(s_{lp}, v_{lp}), B_{helper} \rangle_{\mathcal{L}_v}$

12: **event** Received $\mathbf{n}-\mathbf{f}$ NEWVIEW messages with shares of $\mathcal{P}(s^*, v^*)$ **do**
13: $\mathcal{P}(s^*, v^*) \leftarrow$ CREATETHRESHOLDSIGN($\mathbf{n}-\mathbf{f}$ distinct δ_R shares)

14: **event** Received $\mathbf{n}-\mathbf{f}$ NEWSLOT messages with shares of $\mathcal{P}(s, v)$ **do**
15: $\mathcal{P}(s, v) \leftarrow$ CREATETHRESHOLDSIGN($\mathbf{n}-\mathbf{f}$ distinct δ_R shares)
16: Let $B_{s+1,v}$ be a block of client transaction yet to be proposed
17: Broadcast $m = \langle \text{PROPOSE}, B_{s+1,v}, s+1, v, \mathcal{P}(s, v) \rangle_{\mathcal{L}_v}$

Figure 8: Leader Role in Streamlined HotStuff-1 + Slotting.

1: **function** SAFESLOT($s, v, \mathcal{P}(s_w, w), B_{helper}$) **do**
2: **if** $s = 1$ **and** $B_{helper} \neq \perp$ **then**
3: **if** $w = v - 1$ **and** $B_{helper}.view = v - 1$ **and** $B_{helper}.slot = s_w + 1$ ▹*Check if B_{helper} is valid* **then**
4: **if** $(v - 1 > v_{lp}$ **or** $(v - 1 == v_{lp}$ **and** $s_w + 1 \geq s_{lp}))$ ▹*lexicographical order* **and** R did not vote for any $B_{s_w+1,v-1}$ different from B_{helper} **then**
5: **return** true
6: **else if** $w > v_{lp}$ **or** $(w == v_{lp}$ **and** $s_w \geq s_{lp})$ **then**
7: **return** true
8: **return** false

Backup role (running at each replica R (including leader)) :

9: **event** Received $\langle \text{PROPOSE}, B_{s,v}, \mathcal{P}(s_w, w), B_{helper} \rangle_{\mathcal{L}_v}$ **do**
10: **if** $\mathcal{P}(s_w, w)$ extends $\mathcal{P}(s_w - 1, w)$ ▹*commit-rule-case1* **then**
11: Execute all transactions up to (incl.) $B_{s_w-1,w}$, add result to global-ledger and respond to clients

12: **else if** $s_w = 1$ **and** $\mathcal{P}(s_w, w)$ extends $\mathcal{P}(s^*, w-1)$ ▹*commit-rule-case2* **then**
13: Execute all transactions up to (incl.) $B_{s^*,w-1}$, add result to global-ledger and respond to clients

14: **if** $(s = s_w + 1$ **and** $v = w)$ **or** $(s = 1$ **and** $v = w + 1)$ **and** predecessor of $\mathcal{P}(s_w, w)$ is in global-ledger ▹*Prefix Speculation rule* **then**
15: **if** local-ledger state conflicts with $B_{s_w,w}$ **then**
16: Rollback local-ledger to the common ancestor
17: Execute all transactions in $B_{s_w,w}$ speculatively, add the result to local-ledger and send the client a response ▹*speculatively execute $B_{s_w,w}$*

18: **if** $((s = 1)$ **or** $(s_w = s - 1$ **and** $w = v$ ▹*Consecutive-Slot-Rule*)) **and** SAFESLOT($s, v, \mathcal{P}(s_w, w), B_{helper}$) **then**
19: $\delta_R \leftarrow$ CREATETHRESHOLDSHARE($\mathcal{P}(s_{lp}, v_{lp}), Hash(B_{s,v})$)
20: Send $\langle \text{NEWSLOT}, s, v, \mathcal{P}(s_{lp}, v_{lp}), \delta_R \rangle_R$ to $\mathcal{L}_v$

21: $s \leftarrow s + 1$ ▹*disable voting for slot s*

22: **event** Upon timeout **do**
23: $\langle \text{NEWVIEW}, v+1, \mathcal{P}(s_{lp}, v_{lp}), \delta_h, H_h \rangle_R$ to $\mathcal{L}_{v+1}$.
24: $v \leftarrow v + 1, s \leftarrow 1$. ▹*disable voting for view v*
25: Call PACEMAKER.COMPLETEDVIEW()

Figure 9: Backup Role in Streamlined HotStuff-1 + Slotting.

the signature-shares of the replicas into a threshold signature to create a certificate $\mathcal{P}(s-1, v)$. *Note:* slots do not expire; hence, these $\mathbf{n}-\mathbf{f}$ messages always form a certificate. The leader proceeds to propose slot $B_{s,v}$.

ProposeVote. If $s = 1$ and replica R receives $B_{1,v}$ containing a certificate $\mathcal{P}(s_w, w)$ and a **non-empty** helper block B_{helper}, R first checks if B_{helper} is valid by checking if $\mathcal{P}(s_w, w)$ is of view $v - 1$ and B_{helper} is $B_{s_w+1,v-1}$ (Line 3). If B_{helper} is valid **and** the highest certificate $\mathcal{P}(s_{lp}, v_{lp})$ known to R is lexicographically not greater than B_{helper} **and** R did not vote for any $B_{s_w+1,v-1}$ different from B_{helper} (Line 4), then R votes for $B_{1,v}$ by sending a NEWSLOT message containing a signature share (Line 20).

In other cases where the proposal contains no helper block, R simply checks if $\mathcal{P}(s_w, w)$ is lexicographically not greater than $\mathcal{P}(s_{lp}, v_{lp})$ (Line 6). If so, R votes for the proposal.

Also, leaders are not allowed to skip slots, and replicas do not vote for $B_{s,v}$ if $s > 1$ and $B_{s,v}$ does not extend $B_{s-1,v}$. Such a rule may be referred as *consecutive-slot rule* (Line 18).

Slot-change. There is no timer for individual slots within a view: given a view v, a replica exits slot s upon receiving a well-formed leader proposal for slot (s, v), which extends $\mathcal{P}(s-1, v)$.

View-change. A lack of progress is detected at the view level (not at the slot level). When the timer for view $v - 1$ expires, a replica R exits view $v - 1$; R uses the pacemaker to synchronize entering to view v and sends a NEWVIEW message containing the δ_h, the highest signature-share it sent, H_h, the hash of the highest block it voted and $\mathcal{P}(s_{lp}, v_{lp})$, the highest certificate it received.

Commit Rule. The same as the streamlined design without *slotting*, Streamlined HotStuff-1 with *slotting* has only one commit rule: replicas follow the prefix commit rule (Definition 4.6) to add a transaction to the global-ledger.

However, as we form a two-dimensional chain with *slotting*, there are two different cases when a replica R learns a new certificate $\mathcal{P}(s_w, w)$ and commits the block extended by $\mathcal{P}(s_w, w)$: (1) $s_w > 1$: commits block $B_{s_w-1,w}$ if $\mathcal{P}(s_w, w)$ extends $\mathcal{P}(s_w - 1, w)$. (Line 10) (2) $s_w = 1$: commits block B_{s^*,v^*} if $\mathcal{P}(s_w, w)$ extends $\mathcal{P}(s^*, v^*)$ and $v^* = w - 1$ (Line 12).

Speculation. Replicas may speculate on a block $B_{s_w,w}$ when it is about-to-be committed according to the prefix commit rule. That is, a replica R can speculate on a block $B_{s_w,w}$ upon receiving a proposal $B_{s,v}$ carrying $\mathcal{P}(s_w, w)$ if $B_{s_w,w}$ is from the immediately preceding slot (Line 14), which means: (1) $s = s_w + 1, v = w$; or (2) $s = 1, v = w + 1$.

Prefix Speculation Execution. The enforcement of the Prefix Speculation rule is similar to the regime without *slotting*: *a replica R can speculate on a block B only if the prefix of $\mathcal{P}(B)$ is committed.*

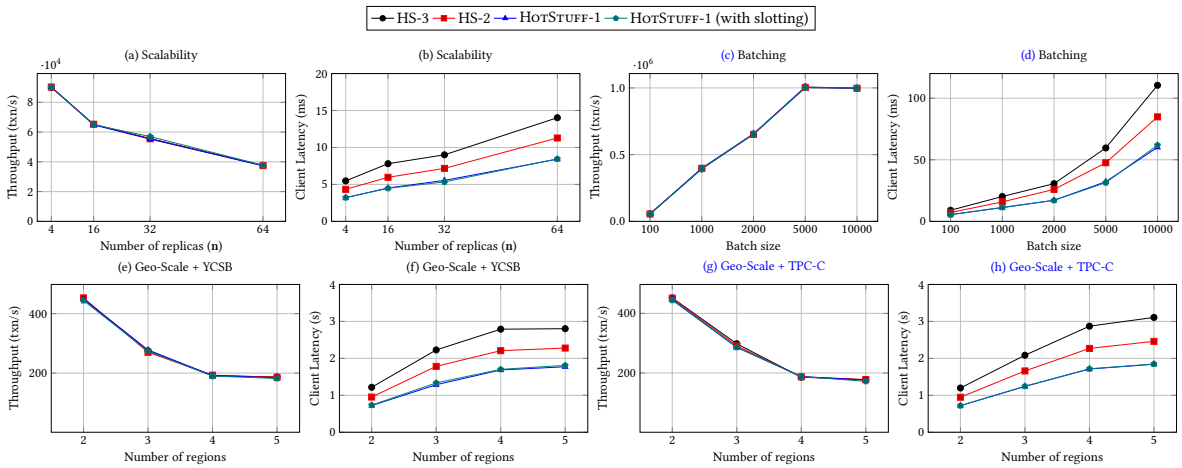

Figure 10: Scalability Plots.

6.2 Advancing at Network Speed

Generally, leaders of BFT consensus must guarantee they extend a highest certificate that all honest replicas will accept (for liveness). A hallmark of protocols in the HotStuff family, often referred to as *(optimistic) responsiveness*, is allowing the protocol to advance *at network speed* unless there are faults. In particular, in HotStuff/HotStuff-2, the leader replacement regime ensures that (after GST), leaders learn the highest certificate without waiting for the pre-determined maximal network delay Δ, unless there is a fault. This feature encompasses two potential paths, happy and unhappy: a new leader waits for the highest locked certificate from the immediately preceding view (highest certificate by definition). If it obtains one, it is in a happy path and the leader can proceed to propose immediately. Otherwise, it is in an unhappy path and it waits until the previous view expires.

Streamlined HotStuff-1 with *slotting* brings a new challenge. That is, $\mathcal{L}_v$ does not know in advance the highest slot s proposed in view $v-1$ because each leader tries to squeeze slots until the very end of its view, and the number of slots in each view is not fixed. Consequently, even in what should be a happy path, i.e., no faults, a (good) leader may nevertheless fail to broadcast its highest certificate for the last slot of his view to all because other replicas have advanced to the next view already.

To guarantee that the first slot proposed by the leader of $\mathcal{L}_v$ extends the highest certificate known to any correct replica from view $v-1$, we introduce the "helper" blocks.

The helper blocks maintain *optimistic responsiveness*: if the leader of view $v-1$ is correct, then the leader of view v can immediately propose $B_{1,v}$ if it can fetch a valid helper block after receiving $\mathbf{n}-\mathbf{f}$ NewView messages. And all correct replicas will vote for $B_{1,v}$ because if any correct replica holds a certificate for a helper block, which is higher than $\mathcal{P}(s_{lp}, v_{lp})$ known to $\mathcal{L}_v$, then at least 1 out of the $\mathbf{n}-\mathbf{f}$ NewView messages contains a vote for the helper block that must be included in the proposal of the first slot.

However, if $\mathcal{L}_{v-1}$ is faulty, then $\mathcal{L}_v$ may be unable to fetch a valid helper block, have no certificates of view $v-1$ (Line 5) or have two conflicting helper blocks (Line 7). Then, $\mathcal{L}_v$ has to wait until it is guaranteed that it has the highest certificate known to correct replicas, which can be achieved through any of the following conditions (Line 9):

(1) a certificate for any the of helper blocks is received; OR
(2) $3\mathbf{f}+1$ NewView messages are received; OR
(3) ShareTimer(v), i.e. 3Δ after $\mathcal{L}_v$ enters view v (the pacemaker guarantees that all correct replicas enter view v within 2Δ, and it takes an extra Δ for NewView messages to be received).

Speculation and Commitment of helper blocks. As it is not guaranteed that there is a certificate of the helper block, we embed the speculation and commitment of helper blocks into those of the first-slot blocks $B_{1,v}$. Replicas speculate or commit a helper block if the respective rule is met for $B_{1,v}$ including it.

7 Evaluation

Our evaluation aims to answer the following:

(1) Scalability of HotStuff-1: throughput and latency with a varying number of replicas and number of transactions in a batch.
(2) Impact of $\mathbf{f}$ additional required responses on HotStuff-1.
(3) Impact of leader-slowness, tail-forking, and rollbacks.

Setup. We use c3.4xlarge AWS machines: 16-core Intel Xeon E5-2680 v2 (Ivy Bridge) processor, 2.8 GHz and 30 GB memory. We deploy up to 64 machines for replicas. Each experiment runs for 120 seconds. We employ *batching* in all our experiments with a default batch size of 100 and mention specific sizes when necessary.

Implementation. We implement all the protocols in Apache ResilientDB (incubating) [13]; C++20 code with Google Protobuf v3.10.0 for serialization and NNG v1.5.2 for networking. Apache ResilientDB is an optimized blockchain framework that provides APIs to implement a new consensus protocol. As threshold signature algorithms are expensive and can quickly bottleneck the computational resources, the leader sends a list of $\mathbf{n}-\mathbf{f}$ digital signatures (from distinct replicas) as a certificate.

Baselines. We compare streamlined HotStuff-1 against two other comparable streamlined protocols:

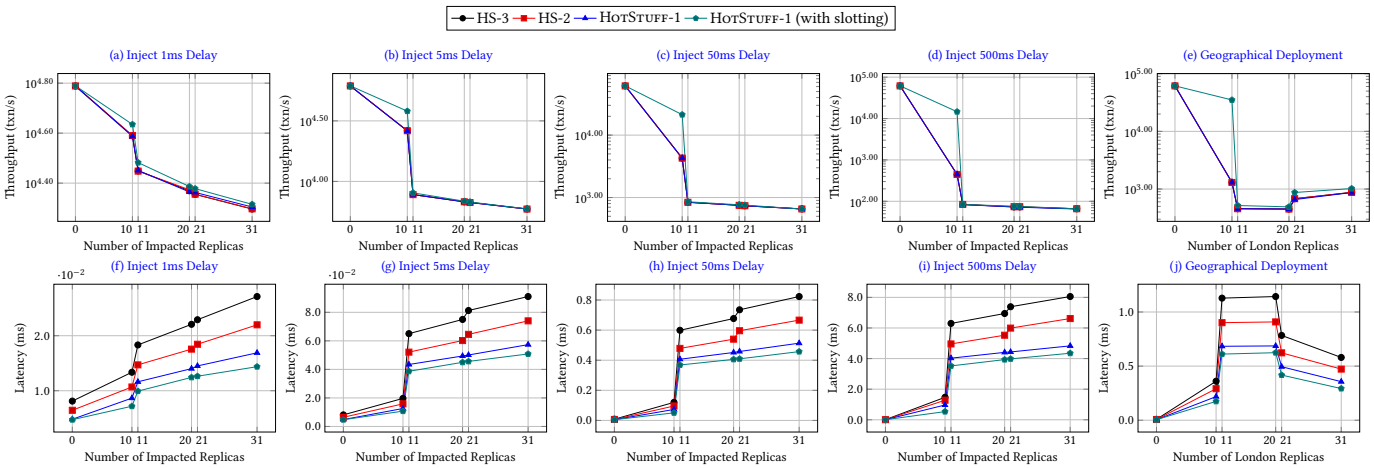

Figure 11: Performance with Varying Network Conditions.

(1) **HotStuff (HS-3).** First streamlined BFT consensus protocol; requires 7 half-phases to reach consensus on a client transaction (total 9 half-phases including client request and response).

(2) **HotStuff-2 (HS-2).** Optimized HotStuff variant that requires 5 half-phases for consensus (total 7 half-phases).

As for HotStuff-1, we implement two versions of it:

(1) **HotStuff-1.** Streamlined BFT consensus protocol with speculative execution that requires 3 half-phases for speculative response (total 5 half-phases).

(2) **HotStuff-1 (with Slotting).**

Workloads. We use two workloads: YCSB [35] and TPC-C [3]:

(1) YCSB. Key-value store write operations that access a database of 600k records.

(2) TPC-C. Online transaction processing (OLTP) operations that access a database of 260k records, simulating a complex warehouse and order management environment.

Unless explicitly stated, we use YCSB as the default workload.

Metrics. We focus on two metrics:

(1) *Throughput* – the maximum number of transactions per second for which the system completes consensus.

(2) *Client Latency* – the average duration between the time a client sends a transaction to the time the client receives a matching quorum of responses ($\mathbf{f}+1$ for HS-3/HS-2 and $\mathbf{n}-\mathbf{f}$ for HotStuff-1) for that transaction.

7.1 Scalability

First, we illustrate the scalability of HotStuff-1.

7.1.1 **Impact of the number of replicas.** In Figures 10 (a) and (b), we present various system metrics as a function of the number of replicas; we increase the number of replicas from $\mathbf{n} = 4$ to $\mathbf{n} = 64$.

Throughput Scalability. As expected, an increase in the number of replicas causes a proportional decrease in the throughput for all the protocols due to an $O(\mathbf{n})$ increased message complexity, which decreases available bandwidth and increases the computational work at each replica. HotStuff-1, with or without slotting, yields the same throughput as HS-3/HS-2 because the message complexity remains the same for all the streamlined protocols.

Latency Scalability. An increase in the number of replicas also causes a proportional increase in the client latency for all the protocols due to an $O(\mathbf{n})$ increased message complexity, which increases the time duration for a leader to collect a quorum of threshold shares and to form a certificate. Moreover, each client needs to wait longer for a larger quorum of messages to arrive. This implies that HotStuff-1 clients should incur higher latency as they must wait for $\mathbf{f}$ more responses. However, HotStuff-1 yields lower latency because speculation guarantees an early finality confirmation. HotStuff-1, with or without slotting, yields 41.5% and 24.2% (for small setups) and 38.5% and 22.7% (for large setups) less client latency in comparison to HotStuff and HotStuff-2.

7.1.2 **Impact of batch size.** Next, in Figures 10 (c) and (d), we increase the number of transactions per batch (batch size) from 100 to 10,000 and run consensus among $\mathbf{n} = 32$ replicas.

For all the protocols, an increase in batch size increases the throughput until the bandwidth or compute is saturated, beyond which throughput will taper off. The gain in throughput at the smaller batch sizes is due to the reduced number of consensus and processing fewer messages. At larger batches, all the protocols become compute-bounded (around batch size 5000) faster than reaching the bandwidth saturation because the gains of reduced consensus are eliminated by the overhead of proposing (for leaders) and processing (for replicas) larger batches. In contrast, the client latency increases with an increase in batch size because it takes a longer time to propose and process a larger proposal in each view.

7.1.3 **Geo-Scale Scalability.** In Figures 10 (e-h), we deploy replicas across the globe; we vary the number of geographical regions from 2 to 5 (North Virginia, Hong Kong, London, San Paulo and Zurich) and uniformly distribute $n = 32$ replicas across these regions.

For these experiments, we test both the YCSB and TPC-C benchmarks. We observe that all protocols present a similar trend for both benchmarks because the round-trip costs between the regions are high, which restricts the maximum throughput that these protocols can yield and increases the latency. As the number of regions

increases from 2 to 5, all the protocols yield up to 59% lower throughput and 46.6% higher latency. However, the trend for throughput and latency remains unsurprisingly the same: HotStuff-1 has the same throughput as the other protocols and incurs least latency.

7.2 Impact of the f Additional Responses

We now experimentally validate our claim: although HotStuff-1 clients wait for **f** additional responses compared to HotStuff/HotStuff-2 clients, HotStuff-1 always yields the lowest latency for clients.

7.2.1 Injecting Message Delay. First, we run an experiment in which we delay messages of a subset of replicas. This experiment illustrates the following: even when more than $\mathbf{f}+1$ replicas suffer high message delays, HotStuff-1 clients do not incur higher latencies. For this experiment, we do the following: (1) Deploy $\mathbf{n}=31$ replicas. (2) As the measured client latencies in earlier experiments for HotStuff-1/HotStuff-2/HotStuff are approximately 5 ms/7 ms/9 ms, we run a series of experiments with increasing injected message delays ($\delta = 1$ ms, 5 ms, 50 ms, 500 ms). (3) Increase the number of impacted replicas (k), where $k = 0, \mathbf{f}, \mathbf{f}+1, \mathbf{n}-\mathbf{f}-1, \mathbf{n}-\mathbf{f}$, and $\mathbf{n}$ (0, 10, 11, 20, 21, 31). We present our findings in Figures 11(a-d) and (f-i).

For all protocols, as the number of impacted replicas increases, there is an increase (decrease) in latency (throughput) because the impacted replicas send/receive messages with a larger delay. This impact is maximal when moving from $k = \mathbf{f}$ (10) to $k = \mathbf{f}+1$ (11) because each certificate formed by a leader now includes at least one impacted replica (certificates need $\mathbf{n}-\mathbf{f}$ signatures). These results also affirm our claim that the true bottleneck for these protocols arises from consensus rather than from sending client responses.

Upon further increasing the number of impacted replicas (from $k = \mathbf{n}-\mathbf{f}-1$ (20) to $k = \mathbf{n}-\mathbf{f}$ (21)), the client latencies of HotStuff/HotStuff-2 increase significantly, while that of HotStuff-1 increases at a normal rate. This is because when $k \geq \mathbf{n}-\mathbf{f}$, clients receive at most **f** responses from non-impacted replicas, and the client latency is determined by the impacted replicas.

Note: When no more than $k = \mathbf{f}$ replicas are impacted, HotStuff-1 with *slotting* yields better performance than all other protocols because slotting allows the non-impacted replicas to propose more blocks during their views.

7.2.2 Geographical Deployment. Next, we deploy $\mathbf{n}=31$ replicas in two geographically distant regions (North Virginia and London); all the clients are in one region (North Virginia). We gradually increase the number of replicas in London (denoted as k), where $k = 0, \mathbf{f}, \mathbf{f}+1, \mathbf{n}-\mathbf{f}-1, \mathbf{n}-\mathbf{f}, \mathbf{n}$. The results in Figures 11(k) and (l) illustrate our findings.

When $k \leq \mathbf{f}$ (10) and $k \geq \mathbf{n}-\mathbf{f}$ (21), leaders in North Virginia and London, respectively, can form a certificate by collecting votes from $\mathbf{n}-\mathbf{f}$ replicas in their region. In contrast, when k is between $\mathbf{f}+1$ (11) and $\mathbf{n}-\mathbf{f}-1$ (20), to form a certificate, each leader needs to wait for at least one message from a replica in the other region, which degrades throughput and latency. Moreover, the case where $k \leq \mathbf{f}$ performs better than the case where $k \geq \mathbf{n}-\mathbf{f}$ because most leaders are co-located with their clients in North Virginia. When $k \leq \mathbf{f}$ or $k \geq \mathbf{n}-\mathbf{f}$, HotStuff-1 with *slotting* yields better performance than all other protocols because slotting allows the leaders with $\mathbf{n}-\mathbf{f}$ co-located replicas to propose more blocks during their views.

7.3 Failure Resiliency

Next, we study the impact of various failures and attacks.

Leader slowness phenomenon. We now study the impact of the leader slowness phenomenon (§6) on different streamlined protocols by varying the number of slow leaders from 0 to **f**; we set $\mathbf{n}=32$ replicas, batch size to 100, and test with two distinct timeout periods: 10 ms and 100 ms. Figure 12 (a)-(d) illustrates our findings.

Unsurprisingly, the slow leaders negatively affect the throughput and client latency for all the protocols but HotStuff-1 (with slotting). In the case of HotStuff-1 (with slotting), each leader has an opportunity to propose multiple batches of transactions, one per slot, which eliminates the need to delay. Larger the timeout period for a leader, the larger the number of batches it can propose.

For example, when the timeout length is 10 ms, with 1 and $\mathbf{f}=10$ slow leaders, HotStuff-1 (with slotting) yields 1.8% and 28.7% lower throughput and 0.9% and 18.5% higher latency than the good case, while other protocols yield 14.5% and 63.5% lower throughput, and 18.7% and 2.8× higher latency. Similarly, when the timeout length is 100 ms, with 1 and $\mathbf{f}=10$ slow leaders, HotStuff-1 (with slotting) yields 3.9% and 34.4% lower throughput, and 5.7% and 27.1% higher latency than the good case, while other protocols yield 63.4% and 94.5% lower throughput and 2.81× and 19× higher latency.

Tail-forking attack. Like the leader slowness phenomenon, the tail-forking attack aims to increase system latency by preventing proposals from correct leaders to commit (§6). In this section, we vary the number of faulty leaders from 0 to **f**; we set $n = 32$ replicas and batch size to 100. In our experiments (Figures 12(e) and (f)), a faulty leader (say view v) ignores the certificate for the proposal of leader of view $v-1$, and instead, extends its proposal from the certificate of the proposal proposed in view $v-2$.

Like earlier, the faulty leaders negatively impact all the protocols but HotStuff-1 (with slotting). In the case of HotStuff-1 (with slotting), each leader has an opportunity to propose multiple batches of transactions and the faulty leader can only skip the last slot.

We observe the following impact due to tail-forking attacks; HotStuff-1 with *slotting* is more resilient to it, especially when the timer length is longer. For example, when the timeout length is 10 ms and 100 ms, with $\mathbf{f}=10$ faulty leader, HotStuff-1 (with slotting) yield 4.1% and 1.4% lower throughput than the good case. In contrast, other protocols yield 31.6% lower throughput than the good case. Similarly, there is minimal change in latency for HotStuff-1 with slotting when compared to the no-failure case. In contrast, with $\mathbf{f}=10$ faulty leaders, other protocols yield up to 45.3% higher client latency than their latency under no-failures.

Rollback. In HotStuff-1, speculation on uncommitted transactions can lead to the cases where a replica may have to roll back the speculated transactions. In Figures 12 (g)-(h), we vary the number of faulty leaders from 0 to f and allow these leaders to force up to **f** correct replicas to roll back transactions; we set $n = 32$ replicas. Notice that in HotStuff-1 with slotting, a faulty leader $\mathcal{L}_v$ can only force the correct replicas to roll back the last slot of the preceding view $v-1$; a faulty-leader cannot skip any slot in its view.

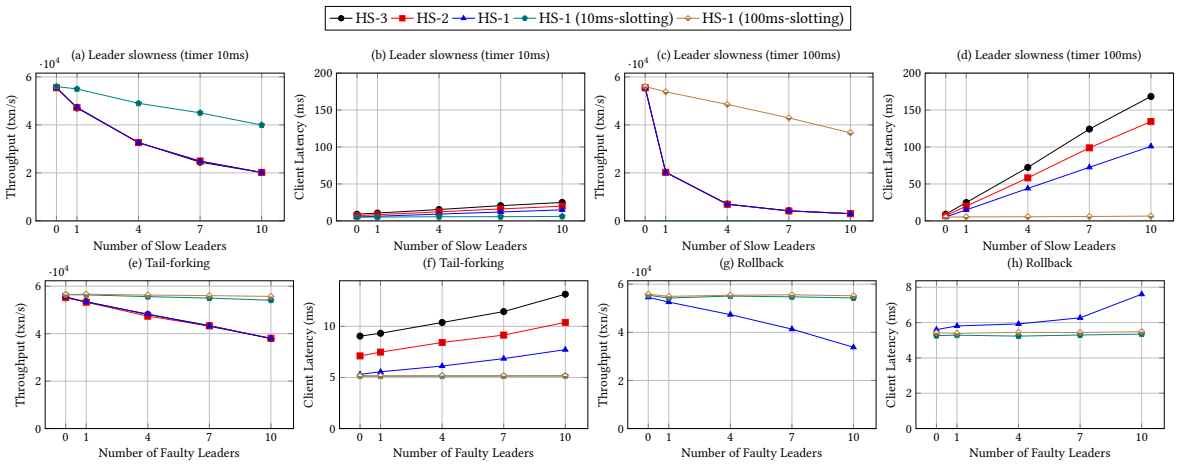

Figure 12: Impact of varying the number of faulty replicas (leader slowness, tail-forking and rollback).

Unsurprisingly, the faulty leaders negatively impact the throughput and latency for HotStuff-1 without slotting. We observe that rollback attacks have minimal impact on HotStuff-1 with slotting. In contrast, HotStuff-1 (without slotting), with $\mathbf{f} = 10$, yields 38.1% lower throughput and 35.8% higher latency than the no-failure case.

8 Related Work

Extensive literature exists on consensus and primary-backup consensus, with numerous studies (e.g., [9, 10, 12, 14, 16, 17, 20, 25, 34, 40, 47, 59, 64, 65, 72, 77, 78, 83, 88, 98]) focused on enhancing consensus systems [21, 22, 43, 46, 48, 49, 51, 58, 60, 66, 75, 79, 82, 85, 97, 99, 99].

Speculation. Protocols belonging to the PBFT family [5, 40, 59] have explored an *optimistic fast-path* approach to speculation. Unfortunately, it works only in fault-free runs and requires a quadratic fallback mechanism. Several papers try to eliminate the dependence on the fast-path, but under leader failures, they also require quadratic fallback mechanisms [4, 42, 50].

Rotational Leader. The HotStuff family of protocols reduces leader-replacement communication costs to linear, enabling regular leader replacement at no additional communication cost or drop in system throughput. HotStuff-2 [67] achieves two-phase latency while maintaining linearity; the published HotStuff-2 algorithm is not streamlined, and streamlined HotStuff-1 contributes a streamlined variant (as well as early finality confirmation). Several other protocols have aimed two-phase streamlined and linear latency. However, Fast-HotStuff [53] and Jolteon [38] have quadratic complexity in view-change; AAR [6] employs expensive zero-knowledge proofs; Wendy [4] rely on a new aggregate signature construction (and are super-linear); Marlin [90] introduces an additional *virtual block*, offering leaders one more chance to propose a block extending the highest certificate that is supported by all correct replicas.

Parallel Proposing. Slotting is complementary to the prior multi-leader protocols like RCC [45], MirBFT [88], SpotLess [54], and DAG protocols [18, 32, 56, 57, 68, 84, 86, 87]. These protocols focus mostly on increasing *throughput*, and the majority have a HotStuff-core. Thus, their designs are orthogonal to this paper. Any reduction in latency, the elimination of leader slowness phenomena, and tail-forking attacks will improve them.

Autobahn presents a data dissemination protocol that separates the task of disseminating client requests from consensus. It allows all replicas, in parallel, to batch and broadcast client requests. However, after dissemination, Autobahn employs PBFT to reach consensus on the execution order for all requests. Thus, Autobahn is orthogonal to the design of HotStuff-1; the PBFT consensus in Autobahn can be replaced with HotStuff-1 to yield lower latency.

Asynchronous Consensus. Leaderless protocols such as HoneyBadger [70] and Dumbo [41] employ a fully decentralized and asynchronous design, which is not directly related to our work. While these protocols claim stronger resilience in asynchronous environments, they also introduce increased complexity.

Leader Slowness. The leader-slowness attack is a well-known problem in blockchains [31, 74, 76]. Prior work has illustrated that in Ethereum, for 59% of blocks, proposers have earned higher MEV rewards than block rewards [74], and any additional delay in proposing can help maximize their MEVs [81]. There are two popular solutions to tackle leader slowness: (i) Exclude any block that misses a set deadline to the main blockchain. However, a clever proposer can still delay proposing until the deadline [15]. (ii) Assign block rewards proportional to the number of attestations; a delayed block will receive fewer attestations and thus reduced block rewards [80]. However, if MEV rewards exceed total block rewards, the proposer makes a profit despite losing any block reward.

Tail-forking attack. As described earlier, BeeGees [39] describes the problem of tail-forking. They present an elegant solution to this problem by requiring replicas to store the proposal sent by the leader and forwarding that proposal in the future rounds. Unfortunately, resending these proposals over the network increases bandwidth consumption.

Real-World Deployments. Several deployed blockchain systems, such as Espresso Systems HotShot [19], Flow Networks [37], and Meter [69] have expressed a latency-over-everything emphasis. Early adopters of HotStuff, DiemBFT [33], and Aptos that uses a

two-phase variant of DiemBFT, Ditto [38], demonstrate the importance of latency. Recently, Spacecoin [2] unveiled plans to launch a trust platform operating within satellite-cubes in orbit, where latency is paramount because the link from Earth to satellites is slow. All of these systems may benefit from incorporating HotStuff-1.

9 Conclusion

The principal goal of this work has been latency reduction for client finality confirmations in streamlined BFT consensus protocols. We demonstrated that HotStuff-1 successfully lowers latency algorithmically via speculation, and furthermore, tackles leader-slowness and tail-forking attacks via slotting. Additionally, we exposed and resolved the *prefix speculation dilemma* that exists in the context of BFT protocols that employ speculation.

References

[1] [n. d.]. Principles for Financial Market Infrastructures (PFMI). https://www.bis.org/cpmi/info_pfmi.htm. Accessed: 2024-12-05.

[2] [n. d.]. Spacecoin Blue Paper. https://github.com/spacecoinxyz/research/blob/main/publications/Blue-Paper-Spacecoinxyz.pdf. Accessed: 2024-12-05.

[3] [n. d.]. TPC-C Benchmark: Standard Specification. https://www.tpc.org/TPC_Documents_Current_Versions/pdf/tpc-c_v5.11.0.pdf. Accessed: 2025-01-16.

[4] Ittai Abraham, Guy Gueta, and Dahlia Malkhi. 2018. Hot-stuff the linear, optimal-resilience, one-message BFT devil. *CoRR, abs/1803.05069* (2018).

[5] Ittai Abraham, Guy Gueta, Dahlia Malkhi, Lorenzo Alvisi, Ramakrishna Kotla, and Jean-Philippe Martin. 2017. Revisiting Fast Practical Byzantine Fault Tolerance. https://arxiv.org/abs/1712.01367

[6] Mark Abspoel, Thomas Attema, and Matthieu Rambaud. 2020. Malicious security comes for free in consensus with leaders. *Cryptology ePrint Archive* (2020).

[7] Yair Amir, Brian Coan, Jonathan Kirsch, and John Lane. 2011. Prime: Byzantine Replication under Attack. *IEEE Trans. Depend. Secure Comput.* 8, 4 (2011), 564–577. https://doi.org/10.1109/TDSC.2010.70

[8] Mohammad Javad Amiri, Divyakant Agrawal, and Amr El Abbadi. 2019. CAPER: A Cross-application Permissioned Blockchain. *Proc. VLDB Endow.* 12, 11 (2019), 1385–1398. https://doi.org/10.14778/3342263.3342275

[9] Mohammad Javad Amiri, Divyakant Agrawal, and Amr El Abbadi. 2021. SharPer: Sharding Permissioned Blockchains Over Network Clusters. In *SIGMOD '21: International Conference on Management of Data*. ACM, 76–88. https://doi.org/10.1145/3448016.3452807

[10] Mohammad Javad Amiri, Chenyuan Wu, Divyakant Agrawal, Amr El Abbadi, Boon Thau Loo, and Mohammad Sadoghi. 2024. The Bedrock of Byzantine Fault Tolerance: A Unified Platform for BFT Protocols Analysis, Implementation, and Experimentation. In *21st USENIX Symposium on Networked Systems Design and Implementation, NSDI 2024, Santa Clara, CA, April 15-17, 2024*, Laurent Vanbever and Irene Zhang (Eds.). USENIX Association, 371–400.

[11] Elli Androulaki, Artem Barger, Vita Bortnikov, Christian Cachin, Konstantinos Christidis, Angelo De Caro, David Enyeart, Christopher Ferris, Gennady Laventman, Yacov Manevich, Srinivasan Muralidharan, Chet Murthy, Binh Nguyen, Manish Sethi, Gari Singh, Keith Smith, Alessandro Sorniotti, Chrysoula Stathakopoulou, Marko Vukolić, Sharon Weed Cocco, and Jason Yellick. 2018. Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains. In *Proceedings of the Thirteenth EuroSys Conference*. ACM, 30:1–30:15. https://doi.org/10.1145/3190508.3190538

[12] Karolos Antoniadis, Antoine Desjardins, Vincent Gramoli, Rachid Guerraoui, and Igor Zablotchi. 2021. Leaderless Consensus. In *41st IEEE International Conference on Distributed Computing Systems*. IEEE, 392–402. https://doi.org/10.1109/ICDCS51616.2021.00045

[13] Apache Software Foundation. 2023. Apache ResilientDB (Incubating). https://resilientdb.incubator.apache.org

[14] Claudio A Ardagna, Marco Anisetti, Barbara Carminati, Ernesto Damiani, Elena Ferrari, and Christian Rondanini. 2020. A Blockchain-based Trustworthy Certification Process for Composite Services. In *2020 IEEE International Conference on Services Computing (SCC)*. IEEE, 422–429. https://doi.org/10.1109/SCC49832.2020.00062

[15] Aditya Asgaonkar. 2021. Proposer LMD Score Boosting, Ethereum Consensus-Specs. https://github.com/ethereum/consensus-specs/pull/2730

[16] Pierre-Louis Aublin, Rachid Guerraoui, Nikola Knezevic, Vivien Quéma, and Marko Vukolic. 2015. The Next 700 BFT Protocols. *ACM Trans. Comput. Syst.* 32, 4 (2015), 12:1–12:45. https://doi.org/10.1145/2658994

[17] Pierre-Louis Aublin, Sonia Ben Mokhtar, and Vivien Quéma. 2013. RBFT: Redundant Byzantine Fault Tolerance. In *2013 IEEE 33rd International Conference on Distributed Computing Systems*. IEEE, 297–306. https://doi.org/10.1109/ICDCS.2013.53

[18] Kushal Babel, Andrey Chursin, George Danezis, Lefteris Kokoris-Kogias, and Alberto Sonnino. 2023. Mysticeti: Low-Latency DAG Consensus with Fast Commit Path. *CoRR* abs/2310.14821 (2023).

[19] Jeb Bearer, Benedikt Bünz, Philippe Camacho, Binyi Chen, Ellie Davidson, Ben Fisch, Brendon Fish, Gus Gutoski, Fernando Krell, Chengyu Lin, et al. 2024. The espresso sequencing network: Hotshot consensus, tiramisu data-availability, and builder-exchange. *Cryptology ePrint Archive* (2024).

[20] Christian Berger and Hans P. Reiser. 2018. Scaling Byzantine Consensus: A Broad Analysis. In *Proceedings of the 2nd Workshop on Scalable and Resilient Infrastructures for Distributed Ledgers*. ACM, 13–18. https://doi.org/10.1145/3284764.3284767

[21] Adithya Bhat, Akhil Bandarupalli, Manish Nagaraj, Saurabh Bagchi, Aniket Kate, and Michael K. Reiter. 2023. EESMR: Energy Efficient BFT - SMR for the masses. In *Proceedings of the 24th International Middleware Conference, Middleware 2023, Bologna, Italy, December 11-15, 2023*. ACM, 1–14. https://doi.org/10.1145/3590140.3592848

[22] Erik-Oliver Blass and Florian Kerschbaum. 2020. BOREALIS: Building Block for Sealed Bid Auctions on Blockchains. In *ASIA CCS '20: The 15th ACM Asia Conference on Computer and Communications Security*. ACM, 558–571. https://doi.org/10.1145/3320269.3384752

[23] Dan Boneh, Ben Lynn, and Hovav Shacham. 2001. Short signatures from the Weil pairing. In *International conference on the theory and application of cryptology and information security*. Springer, 514–532.

[24] Ethan Buchman, Jae Kwon, and Zarko Milosevic. 2018. The latest gossip on BFT consensus. *CoRR* abs/1807.04938 (2018).

[25] Christian Cachin and Marko Vukolic. 2017. Blockchain Consensus Protocols in the Wild (Keynote Talk). In *31st International Symposium on Distributed Computing*, Vol. 91. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 1:1–1:16. https://doi.org/10.4230/LIPIcs.DISC.2017.1

[26] Miguel Castro. 2001. *Practical Byzantine Fault Tolerance*. Ph. D. Dissertation. Massachusetts Institute of Technolog. https://www.microsoft.com/en-us/research/wp-content/uploads/2017/01/thesis-mcastro.pdf

[27] Miguel Castro and Barbara Liskov. 2002. Practical Byzantine Fault Tolerance and Proactive Recovery. *ACM Trans. Comput. Syst.* 20, 4 (2002), 398–461. https://doi.org/10.1145/571637.571640

[28] Pierre Civit, Muhammad Ayaz Dzulfikar, Seth Gilbert, Vincent Gramoli, Rachid Guerraoui, Jovan Komatovic, and Manuel Vidigueira. 2022. Byzantine Consensus Is $\Theta(\mathbf{n}^2)$: The Dolev-Reischuk Bound Is Tight Even in Partial Synchrony!. In *36th International Symposium on Distributed Computing (DISC 2022) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 246)*. Schloss Dagstuhl, 14:1–14:21. https://doi.org/10.4230/LIPIcs.DISC.2022.14

[29] Allen Clement, Edmund Wong, Lorenzo Alvisi, Mike Dahlin, and Mirco Marchetti. 2009. Making Byzantine Fault Tolerant Systems Tolerate Byzantine Faults. In *Proceedings of the 6th USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, 153–168.

[30] James C Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, Jeffrey John Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, et al. 2013. Spanner: Google's globally distributed database. *ACM Transactions on Computer Systems (TOCS)* 31, 3 (2013), 1–22.

[31] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. 2019. Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges. *ArXiv* abs/1904.05234 (2019). https://api.semanticscholar.org/CorpusID:121212213

[32] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. 2022. Narwhal and Tusk: a DAG-based mempool and efficient BFT consensus. In *Proceedings of the Seventeenth European Conference on Computer Systems*. ACM, 34–50. https://doi.org/10.1145/3492321.3519594

[33] Diem. 2020. DiemBFT consensus protocol. https://github.com/diem/diem/tree/latest/consensus

[34] Tien Tuan Anh Dinh, Rui Liu, Meihui Zhang, Gang Chen, Beng Chin Ooi, and Ji Wang. 2018. Untangling Blockchain: A Data Processing View of Blockchain Systems. *IEEE Trans. Knowl. Data Eng.* 30, 7 (2018), 1366–1385. https://doi.org/10.1109/TKDE.2017.2781227

[35] Tien Tuan Anh Dinh, Ji Wang, Gang Chen, Rui Liu, Beng Chin Ooi, and Kian-Lee Tan. 2017. BLOCKBENCH: A Framework for Analyzing Private Blockchains. In *Proceedings of the 2017 ACM International Conference on Management of Data*. ACM, 1085–1100. https://doi.org/10.1145/3035918.3064033

[36] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. 1988. Consensus in the presence of partial synchrony. *J. ACM* 35, 2 (1988), 288–323. https://doi.org/10.1145/42282.42283

[37] Flow. 2025. Flow: The Blockchain for Open Worlds. https://flow.com. Accessed: 2025-01-21.

[38] Rati Gelashvili, Lefteris Kokoris-Kogias, Alberto Sonnino, Alexander Spiegelman, and Zhuolun Xiang. 2022. Jolteon and Ditto: Network-adaptive efficient consensus with asynchronous fallback. In *International conference on financial*

cryptography and data security. Springer, 296–315.

[39] Neil Giridharan, Florian Suri-Payer, Matthew Ding, Heidi Howard, Ittai Abraham, and Natacha Crooks. 2023. BeeGees: Stayin' Alive in Chained BFT. In *Proceedings of the 2023 ACM Symposium on Principles of Distributed Computing* (Orlando, FL, USA) *(PODC '23)*. Association for Computing Machinery, New York, NY, USA, 233–243. https://doi.org/10.1145/3583668.3594572

[40] Guy Golan Gueta, Ittai Abraham, Shelly Grossman, Dahlia Malkhi, Benny Pinkas, Michael Reiter, Dragos-Adrian Seredinschi, Orr Tamir, and Alin Tomescu. 2019. SBFT: A Scalable and Decentralized Trust Infrastructure. In *49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 568–580. https://doi.org/10.1109/DSN.2019.00063

[41] Bingyong Guo, Zhenliang Lu, Qiang Tang, Jing Xu, and Zhenfeng Zhang. 2020. Dumbo: Faster Asynchronous BFT Protocols. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 803–818. https://doi.org/10.1145/3372297.3417262

[42] Suyash Gupta, Jelle Hellings, Sajjad Rahnama, and Mohammad Sadoghi. 2021. Proof-of-Execution: Reaching Consensus through Fault-Tolerant Speculation. In *Proceedings of the 24th International Conference on Extending Database Technology, EDBT 2021, Nicosia, Cyprus, March 23 - 26, 2021*, Yannis Velegrakis, Demetris Zeinalipour-Yazti, Panos K. Chrysanthis, and Francesco Guerra (Eds.). OpenProceedings.org, 301–312. https://doi.org/10.5441/002/edbt.2021.27

[43] Suyash Gupta, Jelle Hellings, and Mohammad Sadoghi. 2019. Brief Announcement: Revisiting Consensus Protocols through Wait-Free Parallelization. In *33rd International Symposium on Distributed Computing (DISC 2019)*, Vol. 146. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 44:1–44:3. https://doi.org/10.4230/LIPIcs.DISC.2019.44

[44] Suyash Gupta, Jelle Hellings, and Mohammad Sadoghi. 2021. *Fault-Tolerant Distributed Transactions on Blockchain*. Morgan & Claypool. https://doi.org/10.2200/S01068ED1V01Y202012DTM065

[45] Suyash Gupta, Jelle Hellings, and Mohammad Sadoghi. 2021. RCC: Resilient Concurrent Consensus for High-Throughput Secure Transaction Processing. In *37th IEEE International Conference on Data Engineering, ICDE 2021, Chania, Greece, April 19-22, 2021*. IEEE, 1392–1403. https://doi.org/10.1109/ICDE51399.2021.00124

[46] Suyash Gupta, Sajjad Rahnama, Jelle Hellings, and Mohammad Sadoghi. 2020. ResilientDB: Global Scale Resilient Blockchain Fabric. *Proc. VLDB Endow.* 13, 6 (2020), 868–883. https://doi.org/10.14778/3380750.3380757

[47] Suyash Gupta, Sajjad Rahnama, Erik Linsenmayer, Faisal Nawab, and Mohammad Sadoghi. 2023. Reliable Transactions in Serverless-Edge Architecture. In *39th IEEE International Conference on Data Engineering, ICDE 2023*. IEEE, 301–314. https://doi.org/10.1109/ICDE55515.2023.00030

[48] Suyash Gupta, Sajjad Rahnama, Shubham Pandey, Natacha Crooks, and Mohammad Sadoghi. 2023. Dissecting BFT Consensus: In Trusted Components we Trust!. In *Proceedings of the Eighteenth European Conference on Computer Systems*. ACM, 521–539. https://doi.org/10.1145/3552326.3587455

[49] Suyash Gupta, Sajjad Rahnama, and Mohammad Sadoghi. 2020. Permissioned Blockchain Through the Looking Glass: Architectural and Implementation Lessons Learned. In *40th International Conference on Distributed Computing Systems*. IEEE, 754–764. https://doi.org/10.1109/ICDCS47774.2020.00012

[50] Jelle Hellings, Suyash Gupta, Sajjad Rahnama, and Mohammad Sadoghi. 2022. On the Correctness of Speculative Consensus. arXiv:2204.03552 [cs.DB] https://arxiv.org/abs/2204.03552

[51] Heidi Howard, Fritz Alder, Edward Ashton, Amaury Chamayou, Sylvan Clebsch, Manuel Costa, Antoine Delignat-Lavaud, Cédric Fournet, Andrew Jeffery, Matthew Kerner, Fotios Kounelis, Markus A. Kuppe, Julien Maffre, Mark Russinovich, and Christoph M. Wintersteiger. 2023. Confidential Consortium Framework: Secure Multiparty Applications with Confidentiality, Integrity, and High Availability. *Proc. VLDB Endow.* 17, 2 (2023), 225–240. https://www.vldb.org/pvldb/vol17/p225-howard.pdf

[52] Dongxu Huang, Qi Liu, Qiu Cui, Zhuhe Fang, Xiaoyu Ma, Fei Xu, Li Shen, Liu Tang, Yuxing Zhou, Menglong Huang, et al. 2020. TiDB: a Raft-based HTAP database. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3072–3084.

[53] Mohammad M Jalalzai, Jianyu Niu, Chen Feng, and Fangyu Gai. 2023. Fast-HotStuff: A fast and robust BFT protocol for blockchains. *IEEE Transactions on Dependable and Secure Computing* (2023).

[54] Dakai Kang, Sajjad Rahnama, Jelle Hellings, and Mohammad Sadoghi. 2024. SpotLess: Concurrent Rotational Consensus Made Practical through Rapid View Synchronization. In *40th IEEE International Conference on Data Engineering, ICDE 2024, Utrecht, Netherlands, May 13-17, 2024*. IEEE.

[55] Jonathan Katz and Yehuda Lindell. 2014. *Introduction to Modern Cryptography* (2nd ed.). Chapman and Hall/CRC.

[56] Idit Keidar, Eleftherios Kokoris-Kogias, Oded Naor, and Alexander Spiegelman. 2021. All you need is dag. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*. 165–175.

[57] Idit Keidar, Oded Naor, Ouri Poupko, and Ehud Shapiro. 2023. Cordial Miners: Fast and Efficient Consensus for Every Eventuality. In *37th International Symposium on Distributed Computing, DISC 2023, October 10-12, 2023, L'Aquila, Italy (LIPIcs, Vol. 281)*, Rotem Oshman (Ed.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 26:1–26:22.

[58] Eleftherios Kokoris-Kogias, Philipp Jovanovic, Nicolas Gailly, Ismail Khoffi, Linus Gasser, and Bryan Ford. 2016. Enhancing Bitcoin Security and Performance with Strong Consistency via Collective Signing. In *Proceedings of the 25th USENIX Conference on Security Symposium*. USENIX, 279–296.

[59] Ramakrishna Kotla, Lorenzo Alvisi, Mike Dahlin, Allen Clement, and Edmund Wong. 2009. Zyzzyva: Speculative Byzantine Fault Tolerance. *ACM Trans. Comput. Syst.* 27, 4 (2009), 7:1–7:39. https://doi.org/10.1145/1658357.1658358

[60] Lucas Kuhring, Zsolt István, Alessandro Sorniotti, and Marko Vukolić. 2021. StreamChain: Building a Low-Latency Permissioned Blockchain For Enterprise Use-Cases. In *2021 IEEE International Conference on Blockchain (Blockchain)*. IEEE, 130–139.

[61] Leslie Lamport. 2001. Paxos Made Simple. *ACM SIGACT News* 32, 4 (2001), 51–58. https://doi.org/10.1145/568425.568433 Distributed Computing Column 5.

[62] Kfir Lev-Ari, Alexander Spiegelman, Idit Keidar, and Dahlia Malkhi. 2019. FairLedger: A Fair Blockchain Protocol for Financial Institutions. In *International Conference on Principles of Distributed Systems*. https://api.semanticscholar.org/CorpusID:182952373

[63] Andrew Lewis-Pye. 2022. Quadratic worst-case message complexity for State Machine Replication in the partial synchrony model. https://arxiv.org/abs/2201.01107

[64] Shengyun Liu, Paolo Viotti, Christian Cachin, Vivien Quéma, and Marko Vukolic. 2016. XFT: Practical Fault Tolerance beyond Crashes. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*. USENIX Association, USA, 485–500.

[65] Dumitrel Loghin, Tien Tuan Anh Dinh, Aung Maw, Chen Gang, Yong Meng Teo, and Beng Chin Ooi. 2022. Blockchain Goes Green? Part II: Characterizing the Performance and Cost of Blockchains on the Cloud and at the Edge. https://arxiv.org/abs/2205.06941

[66] Mads Frederik Madsen, Mikkel Gaub, Malthe Ettrup Kirkbro, and Søren Debois. 2019. Transforming Byzantine Faults using a Trusted Execution Environment. In *15th European Dependable Computing Conference*. IEEE, 63–70. https://doi.org/10.1109/EDCC.2019.00022

[67] Dahlia Malkhi and Kartik Nayak. 2023. Hotstuff-2: Optimal two-phase responsive bft. *Cryptology ePrint Archive* (2023).

[68] Dahlia Malkhi, Chrysoula Stathakopoulou, and Maofan Yin. 2023. BBCA-CHAIN: One-Message, Low Latency BFT Consensus on a DAG. *CoRR* abs/2310.06335 (2023).

[69] Meter.io. 2025. Meter: Decentralized Finance Infrastructure. https://meter.io. Accessed: 2025-01-21.

[70] Andrew Miller, Yu Xia, Kyle Croman, Elaine Shi, and Dawn Song. 2016. The honey badger of BFT protocols. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. 31–42.

[71] Satoshi Nakamoto. 2009. Bitcoin: A Peer-to-Peer Electronic Cash System. https://bitcoin.org/bitcoin.pdf

[72] Faisal Nawab and Mohammad Sadoghi. 2023. Consensus in Data Management: From Distributed Commit to Blockchain. *Found. Trends Databases* 12, 4 (2023), 221–364. https://doi.org/10.1561/1900000075

[73] Diego Ongaro and John Ousterhout. 2014. In search of an understandable consensus algorithm. In *2014 USENIX annual technical conference (USENIX ATC 14)*. 305–319.

[74] Burak Öz, Benjamin Kraner, Nicolò Vallarano, Bingle Stegmann Kruger, Florian Matthes, and Claudio Juan Tessone. 2023. Time Moves Faster When There is Nothing You Anticipate: The Role of Time in MEV Rewards. In *Proceedings of the 2023 Workshop on Decentralized Finance and Security (DeFi '23)*. Association for Computing Machinery, New York, NY, USA, 1–8. https://doi.org/10.1145/3605768.3623563

[75] Sajjad Rahnama, Suyash Gupta, Rohan Sogani, Dhruv Krishnan, and Mohammad Sadoghi. 2022. RingBFT: Resilient Consensus over Sharded Ring Topology. In *Proceedings of the 25th International Conference on Extending Database Technology, EDBT 2022, Edinburgh, UK, March 29 - April 1, 2022*. OpenProceedings.org, 298–311.

[76] Ethereum Roadmap. 2024. Proposer-Builder Separation. https://ethereum.org/en/roadmap/pbs/

[77] Christian Rondanini, Barbara Carminati, Federico Daidone, and Elena Ferrari. 2020. Blockchain-based controlled information sharing in inter-organizational workflows. In *2020 IEEE International Conference on Services Computing (SCC)*. IEEE, 378–385. https://doi.org/10.1109/SCC49832.2020.00056

[78] Pingcheng Ruan, Tien Tuan Anh Dinh, Qian Lin, Meihui Zhang, Gang Chen, and Beng Chin Ooi. 2021. LineageChain: a fine-grained, secure and efficient data provenance system for blockchains. *VLDB J.* 30, 1 (2021), 3–24. https://doi.org/10.1007/s00778-020-00646-1

[79] Vasily A. Sartakov, Stefan Brenner, Sonia Ben Mokhtar, Sara Bouchenak, Gaël Thomas, and Rüdiger Kapitza. 2018. EActors: Fast and flexible trusted computing using SGX. In *Proceedings of the 19th International Middleware Conference*, Paulo Ferreira and Liuba Shrira (Eds.). ACM, 187–200. https://doi.org/10.1145/3274808.3274823

[80] Caspar Schwarz-Schilling. 2022. Retroactive Proposer Rewards. https://notes.ethereum.org/@casparschwa/S1vcyXZL9

[81] Caspar Schwarz-Schilling, Fahad Saleh, Thomas Thiery, Jennifer Pan, Nihar Shah, and Barnabé Monnot. 2023. Time Is Money: Strategic Timing Games in Proof-Of-Stake Protocols. In *5th Conference on Advances in Financial Technologies (AFT 2023) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 282)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 30:1–30:17. https://doi.org/10.4230/LIPIcs.AFT.2023.30

[82] Youren Shen, Hongliang Tian, Yu Chen, Kang Chen, Runji Wang, Yi Xu, Yubin Xia, and Shoumeng Yan. 2020. Occlum: Secure and Efficient Multitasking Inside a Single Enclave of Intel SGX. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '20)*. ACM, 955–970. https://doi.org/10.1145/3373376.3378469

[83] Peiyao Sheng, Gerui Wang, Kartik Nayak, Sreeram Kannan, and Pramod Viswanath. 2021. BFT Protocol Forensics. In *CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 1722–1743. https://doi.org/10.1145/3460120.3484566

[84] Nibesh Shrestha, Rohan Shrothrium, Aniket Kate, and Kartik Nayak. 2024. Sailfish: Towards Improving the Latency of DAG-based BFT. Cryptology ePrint Archive, Paper 2024/472.

[85] Man-Kit Sit, Manuel Bravo, and Zsolt István. 2021. An experimental framework for improving the performance of BFT consensus for future permissioned blockchains. In *DEBS '21: The 15th ACM International Conference on Distributed and Event-based Systems, Virtual Event, Italy, June 28 - July 2, 2021*. ACM, 55–65. https://doi.org/10.1145/3465480.3466922

[86] Alexander Spiegelman, Balaji Arun, Rati Gelashvili, and Zekun Li. 2023. Shoal: Improving DAG-BFT latency and robustness. *arXiv preprint arXiv:2306.03058* (2023).

[87] Alexander Spiegelman, Neil Giridharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. 2022. Bullshark: DAG BFT Protocols Made Practical. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi (Eds.). ACM, 2705–2718.

[88] Chrysoula Stathakopoulou, Tudor David, and Marko Vukolic. 2019. Mir-BFT: High-Throughput BFT for Blockchains. http://arxiv.org/abs/1906.05552

[89] Chrysoula Stathakopoulou, Matej Pavlovic, and Marko Vukolić. 2022. State machine replication scalability made simple. In *Proceedings of the Seventeenth European Conference on Computer Systems*. ACM, 17–33. https://doi.org/10.1145/3492321.3519579

[90] Xiao Sui, Sisi Duan, and Haibin Zhang. 2022. Marlin: Two-Phase BFT with Linearity. In *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 54–66. https://doi.org/10.1109/DSN53405.2022.00018

[91] Florian Suri-Payer, Matthew Burke, Zheng Wang, Yunhao Zhang, Lorenzo Alvisi, and Natacha Crooks. 2021. Basil: Breaking up BFT with ACID (transactions). In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 1–17.

[92] Rebecca Taft, Irfan Sharif, Andrei Matei, Nathan VanBenschoten, Jordan Lewis, Tobias Grieger, Kai Niemi, Andy Woods, Anne Birzin, Raphael Poss, et al. 2020. Cockroachdb: The resilient geo-distributed sql database. In *Proceedings of the 2020 ACM SIGMOD international conference on management of data*. 1493–1509.

[93] Maarten van Steen and Andrew S. Tanenbaum. 2017. *Distributed Systems* (3th ed.). Maarten van Steen. https://www.distributed-systems.net/

[94] Gavin Wood. 2016. Ethereum: a secure decentralised generalised transaction ledger. https://gavwood.com/paper.pdf EIP-150 revision.

[95] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittai Abraham. 2019. HotStuff: BFT Consensus with Linearity and Responsiveness. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*. ACM, 347–356. https://doi.org/10.1145/3293611.3331591

[96] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittai Abraham. 2019. HotStuff: BFT Consensus with Linearity and Responsiveness. In *Proceedings of the ACM Symposium on Principles of Distributed Computing*. ACM, 347–356. https://doi.org/10.1145/3293611.3331591

[97] Rui Yuan, Yubin Xia, Haibo Chen, Binyu Zang, and Jan Xie. 2018. ShadowEth: Private Smart Contract on Public Blockchain. *J. Comput. Sci. Technol.* 33, 3 (2018), 542–556. https://doi.org/10.1007/s11390-018-1839-y

[98] Ce Zhang, Cheng Xu, Jianliang Xu, Yuzhe Tang, and Byron Choi. 2019. GEM^2-Tree: A Gas-Efficient Structure for Authenticated Range Queries in Blockchain. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. IEEE, 842–853. https://doi.org/10.1109/ICDE.2019.00080

[99] Gengrui Zhang, Fei Pan, Sofia Tijanic, and Hans-Arno Jacobsen. 2024. PrestigeBFT: Revolutionizing view changes in BFT consensus algorithms with reputation mechanisms. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 1930–1943.

A Appendix

A.1 Speculation Safety in Basic-HotStuff-1

Allowing replicas to speculatively execute transactions in a proposal m on receiving a certificate for m is not sufficient to guarantee safety. The following example shows that speculatively executing a block after observing a prepare-certificate violates safety.

Assume that the initial state of the system is $\perp$ and the total number of replicas in the system are $\mathbf{n} = 3\mathbf{f} + 1$. Let's divide the $2\mathbf{f} + 1$ correct replicas into three sets: A, A', and A^*, such that $|A| = |A'| = \mathbf{f}$ and $|A^*| = 1$. Further, assume that the first four leaders are faulty.

- The leader of view 1, $\mathcal{L}_1$, proposes B_1 that extends $\mathcal{P}(\perp)$. $\mathbf{n}-\mathbf{f}$ replicas support this proposal by sending their threshold signature-shares for B_1, which allows $\mathcal{L}_1$ to form the prepare-certificate $\mathcal{P}(1)$. $\mathcal{L}_1$ forwards this certificate to only $\mathbf{f}$ correct replicas set A. The replicas in A speculatively execute B_1 and reply to the client.
- Assume the leader of view 2, $\mathcal{L}_2$, ignores the highest known certificate $\mathcal{P}(1)$ and proposes B_2, which extends $\mathcal{P}(\perp)$ to all the replicas. Replicas in sets A' and A^* support B_2, which allows $\mathcal{L}_2$ to form a certificate $\mathcal{P}(2)$. $\mathcal{L}_2$ forwards $\mathcal{P}(2)$ to A' only; A' replicas speculatively execute B_2 and reply to the clients.
- Assume the leader of view 3, $\mathcal{L}_3$, ignores the highest known certificate $\mathcal{P}(2)$ and proposes B_3 that extends $\mathcal{P}(1)$ to all the replicas. Replicas in sets A and A^* support B_3, which allows $\mathcal{L}_3$ to form a certificate $\mathcal{P}(3)$. $\mathcal{L}_3$ forwards $\mathcal{P}(3)$ to A' only; A' replicas roll back B_2, speculatively execute B_3 and its ancestor B_1.
- Assume the leader of view 4, $\mathcal{L}_4$, ignores the highest known certificate $\mathcal{P}(3)$ and proposes B_4 that extends $\mathcal{P}(2)$ to all the replicas. Replicas in sets A and A^* support B_4, which allows $\mathcal{L}_4$ to form a certificate $\mathcal{P}(4)$. Note: for replicas in A, $\mathcal{P}(2)$ conflicts with their highest known certificate $\mathcal{P}(1)$ but as $\mathcal{P}(2)$ has a higher view number, set A replicas have to support. $\mathcal{L}_4$ broadcasts $\mathcal{P}(4)$ to all replicas; A replicas roll back B_1; A' replicas roll back B_1 and B_3; all replicas speculatively execute B_4 and its ancestor B_2 and reply to the clients. Consequently, B_4 gets set as the highest known certificate and will eventually commit.
- Unfortunately, we can have an unsafe situation where the client for B_1 has received $n - f$ responses for the conflicting block B_1 from A, A' and a faulty replica.

This example underscores the Prefix Speculation Dilemma: replicas vote to commit B_v with all its predecessors, but they cannot speculate on the predecessors. The Prefix Speculation rule (Definition 3.1) states that we can allow speculating only when there are no "gaps": when a replica votes to commit B_v, it can speculate on B_v only if B_v extends a committed block B_w. Hence, there are no gaps, and we speculate only on the block in the current view, which is safe. From the existing literature on speculative consensus protocols, we note that Zyzzyva's practice of requiring replicas to send speculation results *carrying a view number* and requiring clients not to mix speculation results from different views can be handy.

A.2 Rollback is Necessary

Providing early finality confirmation is speculative. If a conflicting certificate is formed at a higher view, the local-ledger needs to be rollbacked. We illustrate this in the following scenario.

Assume that the initial state of the system is $\perp$. The leader of view 1, $\mathcal{L}_1$, proposes m that extends $\mathcal{P}(\perp)$. $\mathbf{n} - \mathbf{f}$ replicas support this proposal by sending their threshold signature-shares for m, which allows $\mathcal{L}_1$ to form the prepare-certificate $\mathcal{P}(1)$. $\mathcal{L}_1$ forwards this certificate to $\mathbf{f}$ correct replicas; let us denote this set of replicas A. The replicas in A speculatively execute B_1 in m and reply to the clients. Assume the leader of view 2, $\mathcal{L}_2$, is also faulty; it ignores the highest locked certificate $\mathcal{P}(1)$ and proposes m' that extends $\mathcal{P}(\perp)$ to all the replicas. All but set A replicas (say set A') support m', which allows $\mathcal{L}_2$ to form a certificate $\mathcal{P}(2)$ as there are at least $\mathbf{n} - \mathbf{f}$ replicas in A'. $\mathcal{L}_2$ broadcasts $\mathcal{P}(2)$ to all the replicas. On receiving $\mathcal{P}(2)$, set A replicas will rollback their local-ledger as $\mathcal{P}(2)$ is formed at a higher view than $\mathcal{P}(1)$. Post this, all the correct replicas speculatively execute B_2 and reply to the clients, which mark transactions in B_2 as complete (received $\mathbf{n} - \mathbf{f}$ responses).

A.3 Speculation Safety in Streamlined HotStuff-1not Following Prefix Speculation Rule

The following example shows that speculatively executing a block after observing a two-chain of prepare-certificates in streamlined HotStuff-1 violates safety.

Assume that the initial state of the system is $\perp$ and the total number of replicas in the system are $\mathbf{n} = 3\mathbf{f} + 1$. Let's divide the $2\mathbf{f} + 1$ correct replicas into three sets: A, A', and A^*, such that $|A| = |A'| = \mathbf{f}$ and $|A^*| = 1$.

- The leader of view 1, $\mathcal{L}_1$, proposes B_1 that extends $\mathcal{P}(\perp)$. $\mathbf{n} - \mathbf{f}$ replicas support this proposal by sending their threshold signature-shares for B_1 to $\mathcal{L}_2$, leader of view 2, which forms the prepare-certificate $\mathcal{P}(1)$. Assume the leader of view 2, $\mathcal{L}_2$, proposes B_2 that extends $\mathcal{P}(1)$ and forwards this certificate to only $\mathbf{f}$ correct replicas set A. The replicas in A speculatively execute B_1 and reply to the client (two-chain of certificates: $\mathcal{P}(\perp)$ and $\mathcal{P}(1)$).
- Assume the leader of view 3, $\mathcal{L}_3$ propose B_3 that extends $\mathcal{P}(\perp)$ and send to replicas in set Replicas in sets A' and A^* support B_3, which allows $\mathcal{L}_4$, leader of view 4, to form a certificate $\mathcal{P}(3)$. Assume $\mathcal{L}_4$ propose B_4 that extends $\mathcal{P}(3)$ and send to replicas in set A'; A' replicas speculatively execute B_3 and reply to the clients.
- Assume the leader of view 5, $\mathcal{L}_5$ ignores the highest known certificate $\mathcal{P}(3)$ and propose B_5 that extends $\mathcal{P}(1)$ to all the replicas. Replicas in sets A and $A*$ support B_5, which allows $\mathcal{L}_6$, leader of view 6, to form a certificate $\mathcal{P}(5)$. $\mathcal{L}_6$ forwards $\mathcal{P}(5)$ to A' only; A' replicas roll back B_3, speculatively execute B_5 and its ancestor B_1.
- Assume the leader of view 7, $\mathcal{L}_7$, ignores the highest known certificate $\mathcal{P}(5)$ and proposes B_7 that extends $\mathcal{P}(3)$ to all the replicas. Replicas in sets A and A^* support B_7, which allows $\mathcal{L}_8$ to form a certificate $\mathcal{P}(7)$. Note: for replicas in A, $\mathcal{P}(3)$ conflicts with their highest known certificate $\mathcal{P}(1)$ but as $\mathcal{P}(3)$ has a higher view number, set A replicas have to support. $\mathcal{L}_8$ broadcasts $\mathcal{P}(7)$ to all replicas; A replicas roll back B_1; A' replicas roll back B_1 and B_5; all replicas speculatively execute B_7 and its ancestor B_3 and reply to the clients. Consequently, $\mathcal{P}(7)$ gets set as the highest known certificate and will eventually commit.
- Unfortunately, we can have an unsafe situation where the client for B_1 has received $n - f$ responses for the conflicting block B_1 from A, A' and a faulty replica.

The problem is there is a **gap**. The replicas can vote to commit on B_5, but they cannot speculate on B_1. It is "too late" for them to vote or speculate on ancestors, it would be unsafe. The replicas must not execute/speculate on B_1. We still need a *no-gap* rule (*prefix-commit*) here, allowing speculation on only one block at a time, provided it extends a committed predecessor.

B Correctness Proofs

In this Section, we prove the *safety* and *liveness* of Streamlined HotStuff-1. We first prove the *safety* guarantee.

Lemma B.1. *Let R_i, $i \in \{1, 2\}$, be two correct replicas that executed blocks B_v^i for a given view v. If $\mathbf{n} = 3\mathbf{f} + 1$, then $B_v^1 = B_v^2$.*

Proof. Replica R_i only executes B_v^i after R_i has access to a prepare-certificate for B_v^i in accordance to Figure 5. This prepare-certificate is composed of threshold signature-shares of $\mathbf{n}-\mathbf{f}$ replicas, which we assume cannot be compromised. Let S_i be the replicas that voted for the proposal containing block B_v^i. Let $X_i = S_i \setminus \mathbf{f}$ be the correct replicas in S_i. As $|S_i| = 2\mathbf{f}+1$, we have $|X_i| = 2\mathbf{f}+1-\mathbf{f}$. If $B_v^1 \neq B_v^2$, then X_1 and X_2 must not overlap. Hence, $|X_1 \cup X_2| \geq 2(2\mathbf{f} + 1 - \mathbf{f})$. This simplifies to $|X_1 \cup X_2| \geq 2\mathbf{f} + 2$, which contradicts $\mathbf{n} = 3\mathbf{f} + 1$. Hence, we conclude $B_v^1 = B_v^2$. □

Lemma B.2. *If a replica R receives a certificate $\mathcal{P}(v+1)$ that extends certificate $\mathcal{P}(v)$, then no certificate $\mathcal{P}(w)$ conflicts with $\mathcal{P}(v)$, where view $w > v$, can exist.*

Proof. We know that a replica R received $\mathcal{P}(v+1)$ that extends $\mathcal{P}(v)$, which is only possible if $\mathbf{n} - \mathbf{f} = 2\mathbf{f} + 1$ replicas that set $\mathcal{P}(v)$ as their higher known certificate also voted for $\mathcal{P}(v+1)$. Let's denote the $\mathbf{f} + 1$ correct replicas from these $\mathbf{n} - \mathbf{f}$ replicas as A. Further, certificate $\mathcal{P}(w)$ conflicts with $\mathcal{P}(v)$, $w > v$, which implies that $\mathcal{P}(v)$ and $\mathcal{P}(w)$ extend the same ancestor and $\mathcal{P}(w)$ received support of $\mathbf{n} - \mathbf{f} = 2\mathbf{f} + 1$ replicas. Let's denote the $\mathbf{f} + 1$ correct replicas from these $\mathbf{n} - \mathbf{f}$ replicas as A'. As $w \neq v + 1$, so $w > v + 1$. Moreover, any correct replica that sets $\mathcal{P}(v)$ will not vote for a conflicting block. Thus, $A + A' = 2\mathbf{f} + 2$, which is more than the total number of correct replicas and a contradiction. □

Corollary B.3. *If $\mathbf{f} + 1$ correct replicas speculatively execute a block B_v, then no higher conflicting certificate can commit.*

Proof. From Lemma B.2, we implicitly get this corollary: if $\mathbf{f}+1$ correct replicas speculatively execute a block, then they must have set the certificate for this block as the highest known certificate, and there are not enough correct replicas in the system to vote for a conflicting certificate at a higher view. □

Lemma B.4. *If a correct replica R commits a block B_v, proposed in view v, then no other block can cause it to be rollbacked.*

Proof. Assume block B_w, proposed in view w, $w > v$, conflicts with block B_v and another correct replica R' has committed B_w. This implies that replicas R and R' have conflicting global-ledgers.

For blocks B_v and B_w to commit, R and R' must have followed the commit-rule (§5): R must have received $\mathcal{P}(v+1)$ extending $\mathcal{P}(v)$ and R' must have received $\mathcal{P}(w+1)$ extending $\mathcal{P}(w)$. As $w > v$ and $w \neq v$, so $w > v+1$. From Lemma B.2, we know that once $\mathcal{P}(v)$ and $\mathcal{P}(v+1)$ are formed, then it is impossible to form $\mathcal{P}(w)$. Thus, it contradicts the fact that B_w is committed by R'. □

Corollary B.5. *If a client receives $\mathbf{n}-\mathbf{f}$ responses for block B_v, then no higher conflicting block can be committed.*

Proof. From Lemma B.2 and B.4, we implicitly get this corollary: if a client receives $\mathbf{n}-\mathbf{f}$ responses, then at least $\mathbf{f}+1$ of those must have come from correct replicas. There are only two possible ways for this to happen: (1) At least $\mathbf{n}-\mathbf{f}$ replicas speculatively executed B_v and sent reply to the client. This set of $\mathbf{n}-\mathbf{f}$ replicas includes $\mathbf{f}+1$ correct replicas, and Corollary B.3 tells us that no higher certificate will get formed. (2) At least $\mathbf{f}+1$ replicas executed and committed B_v, which is sufficient to guarantee that B_v cannot be rollbacked (from Lemma B.4). □

Theorem B.6. *(Safety) Streamlined HotStuff-1 guarantees a safe consensus in a system of $n \geq 3\mathbf{f}+1$.*

Proof. Using Lemma B.1, we proved that in Streamlined HotStuff-1, no two correct replicas execute two different blocks for the same view. Further, using Lemma B.4, we prove that a block committed by a replica will never get rollbacked, which guarantees that no two correct replicas can commit conflicting blocks. Consequently, this implies that if a replica R speculatively executes a proposal (say m) based on the Prefix Speculation rule, any proposal that m extends will not be rolled back. Moreover, if the client for m receives $\mathbf{n}-\mathbf{f}$ responses, then m will definitely commit. Thus, we conclude that Streamlined HotStuff-1 guarantees safety. □

Next, we prove the liveness guarantee of streamlined HotStuff-1. Like prior works [67, 95], we assume the existence of GST and an appropriate view timer length τ, which allows correct replicas to overlap in the same view after view synchronization. Such an assumption implies that the view length timer is sufficiently long to allow the leader to process NewView messages, learn the highest known certificate, and propose a block, and for the replicas to vote. We use the notation v_s to denote the first synchronized view after GST.

Lemma B.7. *If a correct replica enters view v, then eventually, all the correct replicas will enter view v.*

Proof. A correct replica exits its current view $v-1$ and moves to the next view v under two conditions: (1) it receives a well-formed Propose message from the leader of the view $v-1$, and post-processing that message, it exits the view. (2) it receives a timeout notification from the pacemaker. Notice that at the start of a pacemaker epoch, all the replicas converge to the same view and set timers for the next $\mathbf{f}$ leaders. Thus, if a correct replica enters view v, then eventually all the correct replicas will timeout and enter view v. □

Lemma B.8. *Assume three consecutive correct leaders $\mathcal{L}_{v+1}$, $\mathcal{L}_{v+2}$ and $\mathcal{L}_{v+3}$, $v+1 \geq v_s$. If $\mathcal{L}_{v+1}$ proposes a block B_{v+1} in view $v+1$, then all correct replicas will commit B_{v+1} in view $v+3$.*

Proof. Recall that the pacemaker facilitates view synchronization, which allows the leader $\mathcal{L}_{v+1}$ to learn the highest certificate known to the correct replicas (say $\mathcal{P}(w)$) once $\mathcal{L}_{v+1}$ receives NewView messages.

$\mathcal{L}_{v+1}$ uses this knowledge to propose block B_{v+1} that extends $\mathcal{P}(w)$. Each correct replica will eventually receive B_{v+1}, will set $\mathcal{P}(w)$ as its highest known certificate (if not already set), and send a NewView message that includes a threshold signature-share in support of B_{v+1} to $\mathcal{L}_{v+2}$. In view $v+2$, $\mathcal{L}_{v+2}$ forms $\mathcal{P}(v+1)$ and proposes B_{v+2} extending the highest certificate $\mathcal{P}(v+1)$. Each correct replica will take the similar steps on receiving B_{v+2}: set $\mathcal{P}(v+1)$ as its highest known certificate and send NewView message voting for B_{v+2} to $\mathcal{L}_{v+3}$. In view $v+3$, $\mathcal{L}_{v+3}$ forms $\mathcal{P}(v+2)$ and proposes B_{v+3} extending the highest certificate $\mathcal{P}(v+2)$. Each correct replica will eventually receive B_{v+3} and set $\mathcal{P}(v+2)$ as its highest known certificate. Post that, all the correct replicas will commit B_{v+1}, in accordance with the *prefix commit rule*. □

Theorem B.9. *(Liveness) All correct replicas eventually commit a transaction T.*

Proof. As there are $\mathbf{n} = 3\mathbf{f}+1$ replicas in total and HotStuff-1 rotates leader in a round-robin fashion, then there is at least one set of three consecutive correct leaders: $\mathcal{L}_{v+1}$, $\mathcal{L}_{v+2}$ and $\mathcal{L}_{v+3}$, $v+1 \geq v_s$. Thus, we can conclude from Lemma B.8, all correct replicas will commit a transaction T proposed in view $v+1$. □

Corollary B.10. *Assume two consecutive correct leaders $\mathcal{L}_{v+1}$ and $\mathcal{L}_{v+2}$, $v+1 \geq v_s$. If $\mathcal{L}_{v+1}$ proposes a block B_{v+1} in view v, then B_{v+1} will eventually get committed.*

Proof. Assume we follow Lemma B.8 to stop at two consecutive correct leaders: $\mathcal{L}_{v+1}$ and $\mathcal{L}_{v+2}$. All the correct replicas will eventually receive, in view $v+2$, block B_{v+2} that extends $\mathcal{P}(v+1)$ and will set $\mathcal{P}(v+1)$ as their highest known certificate. This ensures that at no higher view, a certificate that conflicts with $\mathcal{P}(v+1)$ can exist.

Recall that after the pacemaker's view synchronization, any correct leader $\mathcal{L}_w$ in view $w \geq v_s$ learns the highest certificate before proposing block B_w. Since no certificate at a higher view can conflict with $\mathcal{P}(v+1)$, then each block B_w, where view $w > v+1$, has certificate $\mathcal{P}(v+1)$ as an ancestor (transitively extends). Further, Theorem B.9 proves that there will be at least one set of three consecutive correct leaders (say $\mathcal{L}_w$, $\mathcal{L}_{w+1}$ and $\mathcal{L}_{w+2}$) and when block proposed by $\mathcal{L}_w$ commits, all the ancestors including B_{v+1} will commit. □